

Development of BiLSTM deep learning model to detect URL-based phishing attacks

Öznur Şifa Akçam^{a,b,*}, Adem Tekerek^c, Mehmet Tekerek^d

^a Computer Engineering Department, Graduate School of Natural and Applied Sciences, Gazi University, Ankara, Turkey

^b Computer Engineering Department, Engineering Faculty, Cankiri Karatekin University, Cankiri, Turkey

^c Computer Engineering Department, Technology Faculty, Gazi University, Ankara, Turkey

^d Computer Education and Instructional Technology Department, Educational Sciences Faculty, Ankara University, Ankara, Turkey

ARTICLE INFO

Keywords:

BiLSTM
character-based extraction
deep learning
phishing detection
skip-gram
word-based extraction

ABSTRACT

Phishing attacks steal critical information by exploiting security vulnerabilities in information systems. This study aims to detect URL-based phishing attacks. In this study, a deep learning model based on character and word-based feature extraction is developed. With the developed model, URLs are classified as legitimate or phishing. Bidirectional Long Short-Term Memory (BiLSTM) algorithm and GramBeddings, Malicious and Benign URLs, and Ebbu2017 Phishing datasets were used to develop the model. Also, Mendeley Data Web Page Phishing Detection datasets were used to test the developed model. The developed model achieved test results of 98.24% accuracy and 0.9977 area under curve (AUC) for the GramBeddings dataset, 99.32% accuracy and 0.9986 AUC for the Malicious and Benign URLs dataset, 98.34% accuracy and 0.9981 AUC for the Ebbu2017 dataset, and 90.33% accuracy and 0.9694 AUC for the Mendeley Data Web Page Phishing Detection dataset. These results prove the effectiveness of the model in detecting phishing attacks. The model's uniqueness is that it analyses the structural patterns of URLs through character-based inference and evaluates the contextual meaning through word-based inference. This enables effective detection of phishing URLs at both character and word levels.

1. Introduction

The widespread use of the Internet has led to an increase in data stored on networked devices. Attackers target these data because they are of high importance. Attackers use various methods to obtain the data they want to exploit, including distributing spoofed emails, Short Message Service (SMS) and website links. These methods are designed to appear legitimate by mimicking communications from trusted sources. Clicking on the link in the messages sent to the victim by the attacker causes the victim's passwords, bank account details and personal information to be accessed. This is commonly called a phishing attack, where messages such as emails, Uniform Resource Locator (URL), and SMS are used as bait to trap users [1,2].

The report from the Anti-Phishing Working Group (APWG) for the fourth quarter of 2023 states that there were 4987,809 phishing attacks in 2023 [3]. Data presented in reports published by the APWG show that the incidence of phishing attacks is increasing and that

* Corresponding author.

E-mail addresses: osifa.akcam@gazi.edu.tr, oznursifaakcam@karatekin.edu.tr (Ö.Ş. Akçam), atekerek@gazi.edu.tr (A. Tekerek), mtekerek@ankara.edu.tr (M. Tekerek).

<https://doi.org/10.1016/j.compeleceng.2025.110212>

Received 8 September 2024; Received in revised form 24 January 2025; Accepted 18 February 2025

Available online 28 February 2025

0045-7906/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Internet users are at risk. To protect personal information from unauthorised access, it is crucial to detect phishing attacks before they happen [4]. Various phishing detection methodologies have been developed to combat phishing attacks, including list-based, heuristic, machine learning-based, and hybrid approaches. In list-based methods, lists designated as white and black lists are employed. The white list contains the address information of legitimate websites. The blacklist includes the address information of phishing websites [5]. The address information of websites is stored as a URL. URL is a standardised format for representing the address of a website in a web browser. In list-based approaches, the objective is to identify malicious URLs by matching them with pre-existing URL information in white and black lists. List-based approaches cannot detect the presence of new phishing URL addresses, which necessitates the development of alternative detection methods [6]. More advanced methods have been proposed in response to this limitation, encompassing both heuristic and machine learning-based approaches. The advent of deep learning, in conjunction with advances in natural language processing (NLP), has facilitated innovative solutions for phishing detection. Hybrid methods, which combine two or more methods, have begun to be developed [7]. In the heuristic method, the most common features of phishing website addresses are tried to be revealed. In machine learning, deep learning, and hybrid methods, training is performed with a specific dataset to ascertain whether the URL address corresponds to a phishing or legitimate website [8].

The increasing number and variety of phishing attacks have led to data security breaches. These attacks pose an increasingly significant threat to user security. The shortcomings of existing phishing detection methods emphasise the need for more innovative and effective solutions. This study aims to accurately classify website URLs as phishing or legitimate addresses by developing a deep learning model based on character and word-based feature extraction. The reason for combining character and word-based feature extraction methods is to overcome the inadequacies of a single method and provide more accurate results. This approach provides a more comprehensive intrusion detection analysis by evaluating characteristic patterns and textual content features. The URL address is a textual expression comprising a sequence of characters, as illustrated in Fig. 1. Identifying whether the URL address corresponds to a legitimate or phishing website is a text classification problem in NLP [2]. To explore this problem in greater depth, the study proposes and examines specific research questions. This study aims to ascertain whether the URL address transmitted is a fraudulent URL address used for phishing attacks or a genuine URL address. For this purpose, a deep learning model based on character and word-based feature extraction is proposed to classify URLs. In the context of this study, the responses to the following research questions (RQ) were explored to develop a solution to the challenge of phishing classification. **RQ1.** What are the distinctions between character-based feature extraction and word-based feature extraction in classifying different URL addresses? What is the effect of combining these feature extraction methods on classification performance? **RQ2.** What are the effects of the number of samples on feature extraction and classification success in deep learning algorithms in data sets with different sample numbers? **RQ3.** How does an imbalanced label distribution in the dataset affect the classification performance of feature extraction and deep learning algorithms?

This study contributes to the literature in several key ways. The main contributions of the study to the existing literature are as follows. **(Innovative Approach)** This paper presents an innovative approach that combines character and word-based feature extraction methods. This combination comprehensively represents URLs' structural and content aspects, enabling the classification model to work with a rich feature set. **(Effective Integration)** Character-based feature extraction is an effective method for representing the structural features of URLs. In contrast, word-based feature extraction is an effective method for reflecting their semantic content. Integrating these two methods enhances the model's capacity to recognise and generalise across different types of URLs more effectively. **(Advanced Detection Capabilities)** The integration of character and word-based methods allows for character and word-level analysis of URLs, thereby enhancing the model's comprehensive detection capabilities. This approach facilitates the identification of phishing patterns that may not be discernible through applying a single method. It improves accuracy and error tolerance and effectively detects more complex phishing URLs. **(Experimental Success)** Tests conducted with a unique dataset of shortened URLs revealed that the proposed model also demonstrated successful performance with this URL type. **(Reduced Dependency on Manual Features)** The proposed model eliminates the need for manual feature extraction and expert feature identification, streamlining the detection process. **(Language-Independent Classification)** The proposed method offers a language-independent classification approach that does not rely on third-party services.

The rest of this paper is organised as follows: Section 2 reviews the existing literature in the field. Section 3 describes the datasets and algorithms used. Section 4 gives a detailed explanation of the proposed methodology. Section 5 analyses the results of the experimental phase. Section 6 compares these results with previous work, and Section 7 concludes the paper.

2. Related Works

Feature engineering based on URL character combinations is used to detect phishing attacks. For this purpose, distinguishing features in textual expressions are identified, and wrapper, filter, and embedded methods are applied for feature extraction. In particular, wrapper and filter methods, which require manual feature extraction, are commonly used with traditional machine-learning algorithms [7]. In deep learning and natural language processing fields, textual content is converted into mathematical sequences, enabling analysis of URLs through character-based and word-based methods. These sequences form the input layer for deep

<https://www.abc.com/1234?xyz=1a3bc#45%de@13>

Fig. 1. Sample URL address.

learning algorithms [9].

Sahingöz et al. [2] classified URLs as phishing or legitimate using algorithms such as Naive Bayes, Random Forest, k-Nearest Neighbors (kNN), Adaboost, and Decision Trees. With a self-created dataset (Ebbu2017 Phishing Dataset) comprising 40 NLP-based and 1701-word vector features, the Random Forest algorithm achieved the highest accuracy at 97.98%. Word to Vector (Word2Vec) is a deep learning model that converts words into numerical vectors, forming the basis for embedding layers [9]. Other studies, such as [10] and [11], have also employed embedding methods like Word2Vec to convert URLs for classification by deep learning algorithms. For instance, Wang et al. [10] combined Recurrent Neural Network (RNN) and Convolutional Neural Network (CNN) in their PDRCNN model using datasets from Alexa and PhishTank, achieving an accuracy of 97%. In addition to Word2Vec, techniques such as Bidirectional Encoder Representations from Transformers (BERT) and Global Vectors for Word Representation (GloVe) are also used to generate vector representations from text, addressing various challenges in NLP, particularly for text classification and language modelling. GloVe and BERT are analogous word embedding techniques [12,13]. Benavides-Astudillo et al. [12] utilised GloVe by extracting words from web pages' Hyper Text Markup Language (HTML) code and converting them into vector representations for input into Long Short-Term Memory (LSTM), BiLSTM, Gated Recurrent Unit (GRU), and Bidirectional Gated Recurrent Unit (BiGRU) algorithms, achieving 97.39% accuracy with BiGRU. Meanwhile, Elsadig et al. [13] applied BERT for word-based feature extraction from the Phishing Site Predict dataset, classifying URL addresses with a CNN algorithm, which resulted in an accuracy of 99.66%.

Yang et al. [14], URL addresses are converted into character strings using the char embedding method. After this conversion, distinctive features are extracted using CNN and LSTM algorithms, followed by classification with the softmax function. The softmax output serves as input for a dynamic category decision algorithm. If the CNN-LSTM classification exceeds a specified threshold, detection is output immediately; if not, a second detection step is initiated, combining statistical, webpage code, and text features with deep learning features from the previous step for classification using the XGBoost algorithm. This multidimensional feature fusion method achieved an accuracy of 98.99%, reducing phishing detection time. Similarly, Alshehri et al. [15] utilised a Deep Neural Network (DNN) for classifying phishing URLs, achieving an accuracy rate of 98.13%.

Ozcan et al. [1] proposed a hybrid method using 40 NLP-based features from [2], combining character embeddings and NLP features. URLs were converted into character strings and fed into a BiLSTM model, while the NLP features were used as input for a DNN. Discriminative features from both models were integrated for classification. Among LSTM, CNN, DNN, and RNN, the DNN-BiLSTM hybrid model performed best, achieving accuracy rates of 98.79% and 99.21% on the Ebbu2017 Phishing and PhishTank datasets, respectively. Similarly exploring character-based approaches, n-gram embedding is a standard NLP method for classification, converting text into vectors by creating word or character n-grams. Bozkir et al. [8] applied character-based n-gram embedding, converting URLs into matrices as input for a CNN, followed by a bidirectional LSTM to capture sequential relationships. Attention and fully connected layers were then used to classify phishing versus legitimate URLs. Due to its character-based n-gram embedding, this method does not emphasise features capturing semantic relationships between n-grams. Using their GramBeddings dataset, the researchers achieved an accuracy rate of 98.27%. Some studies employ character and word embedding methods to classify URL addresses for phishing detection. Rasymas et al. [16] used character, word embedding, and lexical features in their research. The outputs from the character and word embedding layers were input to a CNN algorithm and then to a GRU algorithm, with multiple dense layers employed, receiving 60 lexical features. All outputs were integrated into a single classification process, achieving an accuracy rate of 94.4%. Similarly, Zheng et al. [9] utilised URL addresses as input, creating separate word and character embedding matrices combined in a highway network module and fed into a CNN algorithm, resulting in an accuracy of 98.30%.

Smart devices have become increasingly vulnerable to phishing attacks with the widespread use of mobile technologies. Rao et al. [17] addressed this issue by developing an application called PhishDump for mobile devices. Their model uses a multiple ensemble approach, classifying features derived from LSTM with Support Vector Machine (SVM), achieving 97.30% accuracy on their dataset and 98.50% on a benchmark dataset. In exploring phishing attack detection, some studies integrate multiple methodologies. For instance, Xiao et al. [18] generated a synthetic URL dataset using a Generative Adversarial Network (GAN) to create a balanced training set. They classified the data by combining a CNN algorithm with a multi-head self-attention mechanism. They found that the balanced dataset from the GAN yielded superior results to the unbalanced dataset, achieving an accuracy of 95.6%.

A model proposed by Afzal et al. [11] utilised a publicly available dataset from Kaggle and presented a hybrid model named URLdeepDetect, which combines supervised and unsupervised learning algorithms. This model uses LSTM and K-Means clustering for classification, achieving 98.30% and 99.70% accuracy rates, respectively. Similarly, Roy et al. [19] also used a Kaggle dataset and classified the data with LSTM, BiLSTM, and GRU algorithms, achieving the highest accuracy of 99.0% with the BiLSTM algorithm. The studies by [20] and [21] aim to achieve high accuracy in phishing URL detection using deep learning models. Aslam et al. [20] developed a two-phase AntiPhishStack model that employs character-level features and achieved an accuracy rate of 96.04% on datasets from Alexa, Common Crawl, Yandex, and PhishTank using a stacked approach with XGBoost as the final classifier. Zaimi et al. [21], on the other hand, addressed data imbalance through permutation feature importance and the SMOTE-Tomek method, achieving accuracy rates between 94.12% and 97.82% on the Mendeley dataset using XGBoost, CNN, LSTM, and hybrid models. Zaimi et al. [21] also used a dataset with pre-extracted URL features. Both studies highlight high accuracy in phishing detection while noting challenges related to data imbalance and feature limitations.

Current studies in the literature have generally focused on either character-based feature extraction or word-based feature extraction. Character feature extraction has been utilised independently or in conjunction with predefined features. The continuous evolution of phishing attacks raises questions about the reliability of these pre-extracted features. Consequently, it has become critically necessary for new models to identify unique features based on URL addresses. However, the number of studies employing character and word-based feature extraction is limited. For instance, Rasymas et al. [16] incorporated lexical features along with character and word-based features; however, it has been observed that they do not possess significant discriminative power in phishing

detection. In the study by Zheng et al. [9], the imbalance in the dataset presents a significant limitation, as the number of legitimate URLs exceeds that of phishing URLs, negatively affecting the proposed model's effectiveness. A literature review indicates that character and word-based feature extraction have been used separately, yet there are insufficient studies on their combined usage. This study combines character and word-based feature extraction methods to address this gap and conducts experimental observations on four datasets. This innovative approach emphasises the uniqueness of this work compared to other studies in the literature and demonstrates its potential to provide more effective solutions for phishing detection.

Phishing attacks present a serious risk to information and computer systems by exploiting security gaps and vulnerabilities within these systems. Although studies in the literature have contributed to the detection of phishing attacks, the number and complexity of phishing attacks are increasing due to the development of new methods. Enhancing the defensive techniques employed to detect phishing attacks is similarly necessary. In this study, four distinct datasets were used to monitor phishing attacks. The datasets comprise a disparate number of URL addresses. As URL addresses are composed of various characters, utilising raw data with deep learning algorithms is impossible. It is necessary to express the data in a mathematical format. Character and word-based methods were employed concurrently in the data preprocessing and feature extraction stages. Two distinct sets of features were derived for characters and words, respectively. The extracted features comprise high-dimensional integers. An embedding layer was employed to transform the features into lower-dimensional and dense vectors. The embedding layer's output contributes to the model's effective learning by creating more meaningful and efficient inputs for deep learning algorithms. The character and word-based extracted features were trained separately using the BiLSTM algorithm, supported by the Attention layer. The extracted characters and word-based features from the training were combined, and a classification was performed. A comparison of the results obtained with those of previous studies in the literature reveals that this method of defence can be used to control phishing attacks that are constantly developing. Within this scope, combining character-based and word-based feature extraction methods provides a versatile and comprehensive approach to detecting phishing URLs. The character-based method can capture subtle patterns and anomalies by examining each character of URLs, whereas the word-based method focuses on the semantic relationship between words. This combination allows for analysing phishing URLs at both the character and word levels, facilitating a more comprehensive detection process. To illustrate, a phishing URL that evades detection by one method may be identified by the other, thereby enhancing the model's resilience to errors. Furthermore, the integration of two distinct levels of methods facilitates the detection of more advanced and intricate phishing attacks. This strategy yields enhanced accuracy and dependability in identifying phishing URLs, thus reinforcing the security of users and the reliability of the Internet.

3. Material and Methods

This section of the paper briefly describes the problem statement, the datasets, and the algorithms used in the study.

3.1. Problem Definition

The classification of phishing and legitimate URL addresses can be considered a text classification problem. To illustrate, consider a data set of URL addresses and their associated labels. These are presented in the form of the Eq. (1).

$$D = \{(a_1, b_1), (a_2, b_2), \dots, (a_n, b_n)\} \quad (1)$$

In Eq. (1), D represents the dataset. The variable $\{a_1, a_2, \dots, a_n\}$ represents the URL addresses. The variable $\{b_1, b_2, \dots, b_n\}$ represents the label of the URL addresses. The variable b can assume two distinct values $\{0, 1\}$. The value 0 ($b = 0$) signifies that the URL address in question belongs to the legitimate class, whereas the value 1 ($b = 1$) indicates that the URL address in question belongs to the phishing class. To be utilised in deep learning algorithms, URL addresses comprising a variety of characters must be converted into vectors. In Eq. (2), a_t represents the URL, while x_t represents the feature vector obtained from the URL address. The URL address a_t is transformed into x_t .

$$a_t \rightarrow x_t \quad (2)$$

Eq. (3) illustrates the function f traversing from the space R^n (the set of n -dimensional real numbers) to the space R (the set of real numbers). R^n represents the set of definitions of the function f , whereas R represents the set of values.

$$f: R^n \rightarrow R \quad (3)$$

In order to ascertain whether the URL addresses in question belong to the phishing (1) or legitimate (0) classes, the function f makes a score prediction. This prediction is shown in Eq. (4). Since the value of b takes the values 0 and 1, the prediction is performed with the sigmoid function. The sigmoid function takes values between 0 and 1. In Eq. (4) and Eq. (5), σ represents the sigmoid function. In Eq. (4), $f(x_t)$ represents the prediction value of the classification result of the model. $\hat{b}_t$ is the predicted value of the URL address in instance t (Eq. (4)). The threshold value is set at 0.5. If the result of the sigmoid function is equal to or greater than 0.5, the URL address is classified as phishing. If the result of the sigmoid function is less than 0.5, the URL address is classified as legitimate [11,14].

$$\hat{b}_t = \sigma(f(x_t)) \quad (4)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (5)$$

3.2. Datasets

To address the problem of URL phishing classification, this study utilised four publicly available datasets with varying numbers of URLs. The first dataset is the GramBeddings Dataset [8], which contains 799,991 URLs evenly distributed, with 50% phishing and 50% legitimate URLs. The second dataset, the Malicious and Benign URLs Dataset from Kaggle [22], comprises 450,176 URLs, with an imbalanced distribution of 23.2% phishing and 76.8% legitimate URLs. The third dataset, the Ebbu2017 Phishing Dataset [2], includes 73,575 URLs, with a nearly balanced distribution of 50.5% phishing and 49.5% legitimate URLs. The final dataset is the Mendeley Data Web Page Phishing Detection Dataset [23], which contains 11,430 URLs, with 50% phishing and 50% legitimate URLs. The GramBeddings, Ebbu2017 Phishing and Mendeley Data Web Page Phishing Detection datasets are balanced regarding phishing and legitimate classes, while the Malicious and Benign URLs dataset is unbalanced. Table 1 provides a detailed statistical summary of these datasets. The first three datasets were used to develop the model proposed in this study. The fourth dataset, which is smaller in size, was included to allow for the testing of alternative methods without memory issues and to evaluate the model's performance on a smaller dataset.

The GramBeddings, Malicious and Benign URLs, Ebbu2017 Phishing, and Mendeley Data Web Page Phishing Detection datasets were chosen for several reasons. First, they are all publicly accessible. Second, they contain varying numbers of phishing and legitimate URLs, with three balanced datasets and one imbalanced distribution. These factors make the datasets suitable for addressing the URL phishing classification problem.

3.3. Algorithms

3.3.1. Word2Vec

Word2Vec is a natural language processing model that uses logic similar to artificial Neural Net.. The model is used to transform words into numerical vectors. Vectors represent the words within the texts. The meanings of the words are stored numerically in the vectors. Furthermore, the vectors representing words with similar meanings exhibit high mathematical proximity [24]. In the Word2Vec model, the semantic proximity between words is expressed in numerical form. The conversion of words into vectors enables the application of deep learning algorithms. The Word2Vec algorithm employs two distinct architectural approaches: Continuous Bag of Words (CBoW) and skip-gram. These architectures are used to generate word vectors [11]. Fig. 2 provides a visual representation of the working logic of the CBoW and skip-gram algorithms. As illustrated in Fig. 2, the input data is a text, in this case, the URL address. The 'https://' portion of the URL address is removed. (It is not necessary to remove the 'https://' portion of the URL address; this does not cause any issues. The 'https' is also treated as a word.) The text is divided into individual words, or tokens, according to the presence of specific characters, such as '/', '.', and '-'. To illustrate this process, we will assume that the target word is 'gazi'. The words 'tf', 'bm', 'edu', and 'tr' are identified as context words. Word2Vec incorporates a parameter designated as the window size. The parameter in question determines the number of words to be searched from the left and right sides of the target word. Context words are taken as input data to create word vectors using the CBoW algorithm, and the target word is attempted to be predicted. To illustrate, if the window size value is set to 2, the words 'tf', 'bm', 'edu', and 'tr' are designated as input and transformed into vectors. A mean vector is created, and the softmax function (Eq. (6)) is employed to predict the target word, 'gazi'. The Skip-gram algorithm functions in a manner that is the inverse of the CBoW algorithm. The target word is employed as input data. A target word vector is then created, and context words are subsequently predicted. The softmax function (Eq. (6)) is utilised in the prediction process. The difference between the expected context words and the actual context words is minimised. Consequently, the semantic relations between words are identified mathematically.

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad (6)$$

Table 1
Statistical analysis of URL datasets.

	GramBeddings Dataset [8]	Malicious and Benign URLs Dataset [22]	Ebbu2017 Phishing Dataset [2]	Mendeley Data Web Page Phishing Detection Dataset [23]
Total number of URLs	799.991	450.176	73.575	11.430
Number of phishing URLs	399.998 (50%)	104.438 (23.2%)	37.175 (50.5%)	5715 (50%)
Number of legitimate URLs	399.993 (50%)	345.738 (76.8%)	36.400 (49.5%)	5715 (50%)
Maximum length of the URL	7833	2314	3992	1633
Minimum length of the URL	9	8	9	5
Average length of the URL	58.7934	52.4561	61.8635	53.7309

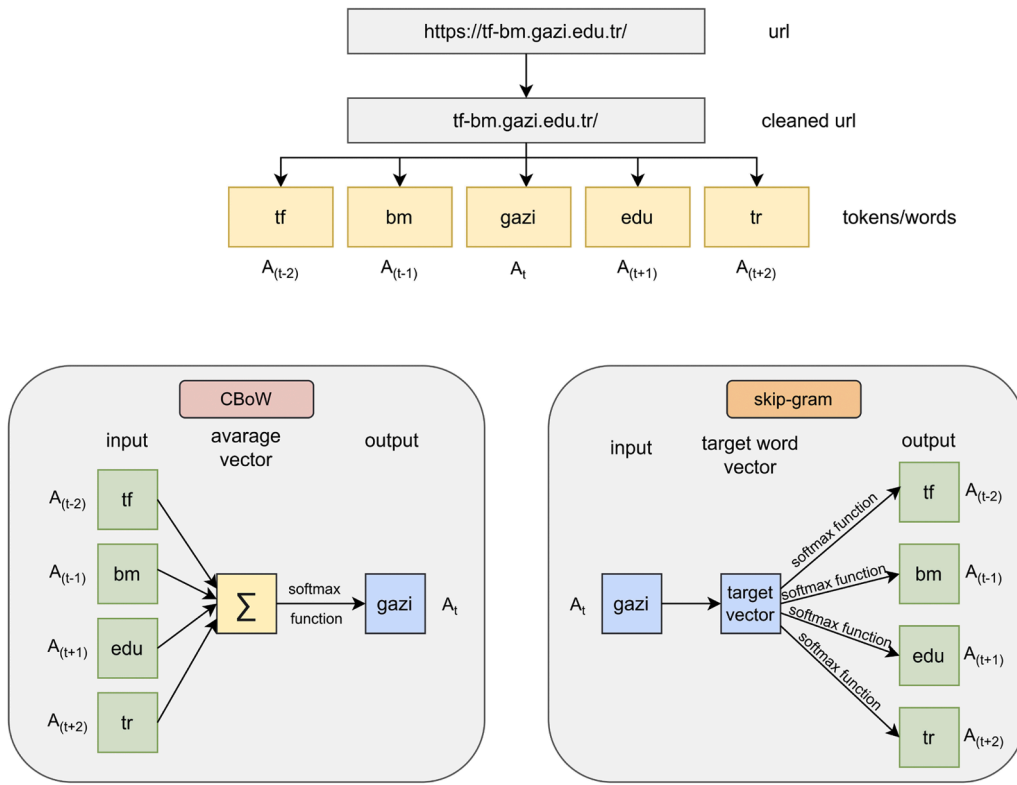


Fig. 2. Word2Vec algorithms: CBoW and skip-gram.

3.3.2. LSTM

LSTM functions as a recurrent neural network model, explicitly targeting the issue of vanishing gradients prevalent in RNNs. This issue arises particularly in deep forward propagation networks, where adding layers impedes practical training. LSTMs address the limitations of RNNs concerning short-term dependencies while facilitating the learning of long-term dependencies [7]. The LSTM architecture comprises essential components: the cell state, forget gate, input gate, and output gate, as depicted in Fig. 3. The cell state retains long-term memory and manages information storage and forgetting. The forget gate controls the retention or removal of information in the cell state. Meanwhile, the input gate governs the integration of new information into the cell state, and the output gate determines the subsequent hidden state [17].

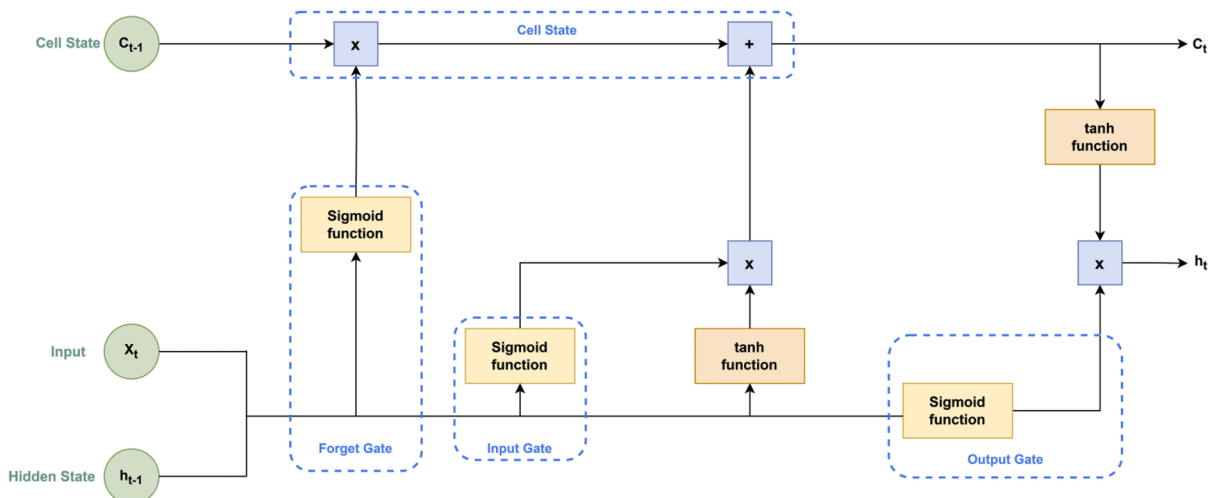


Fig. 3. LSTM architecture.

Let us consider a sequence of words of the form $(w_1, w_2, \dots, w_n)$. Let (h_t, c_t) be the state of the LSTM at the time t . h_t presents the hidden state, while c_t represents the cell state. The LSTM model accepts the input vector (x_t) and the previous state (h_{t-1}, c_{t-1}) as inputs, the forget gate determines which information is discarded from the cell state. The forget gate is calculated using the previous hidden state (h_{t-1}) and the input vector (x_t) (Eq. (7)). For each number in the (c_{t-1}) cell state, a number between 0 and 1 is generated. Based on the generated number, 0 signifies that the information will be discarded, while 1 signifies that the information will be retained. In Eq. (7), f_t represents the output of the forget gate, σ is the result of the sigmoid activation function, w_f stands for the weight matrix, b_f is the bias value, and $[h_{t-1}, x_t]$ is the combination of the hidden state and the input vector.

$$f_t = \sigma(w_f * [h_{t-1}, x_t] + b_f) \quad (7)$$

The input gate is calculated according to the formula in Eq. (8) and Eq. (9). The input gate vector and the new candidate cell state components are included. The i_t The variable represents the input gate output, the $\tilde{c}_t$ The variable represents the new candidate cell state, the w_i and w_c variables represent the weight matrices and the b_i and b_c variables represent the bias values.

$$i_t = \sigma(w_i * [h_{t-1}, x_t] + b_i) \quad (8)$$

$$\tilde{c}_t = \tanh(w_c * [h_{t-1}, x_t] + b_c) \quad (9)$$

The new cell state is calculated by the formula specified in Eq. (10). The outputs of the forget gate and the input gate are employed. The c_t the variable represents the updated cell state, while the c_{t-1} the variable represents the previous cell state.

$$c_t = f_t * c_{t-1} + i_t * \tilde{c}_t \quad (10)$$

The output gate determines which information will become the hidden state by employing the formulas specified in Eqs. (11) and (12). The o_t the variable represents the output of the output gate, h_t the updated hidden state, w_o the weight matrix, and b_o the bias value.

$$o_t = \sigma(w_o * [h_{t-1}, x_t] + b_o) \quad (11)$$

$$h_t = o_t * \tanh(c_t) \quad (12)$$

3.3.3. BiLSTM

The BiLSTM algorithm is based on the principle that the LSTM algorithm functions in both directions. As illustrated in Fig. 4, the BiLSTM algorithm comprises forward and backward LSTM layers. The forward and backward operation of the BiLSTM allows it to consider both previous and subsequent information in sequential data. The forward-backwards working principle enables the algorithm to capture a greater degree of context information. Consequently, it is capable of making more comprehensive and accurate predictions than LSTM. The mathematical formulae of the BiLSTM algorithm are identical to those of the LSTM algorithm [19,25].

3.3.4. CNN

CNN is a deep learning architecture widely used in fields such as image processing, computer vision, and disease detection from medical images. Its basic structure comprises convolution, pooling, and fully connected layers (Fig. 5). CNNs are particularly effective when the dataset consists of images, as they can directly process images as input. The convolution layer applies filters (kernels) to the

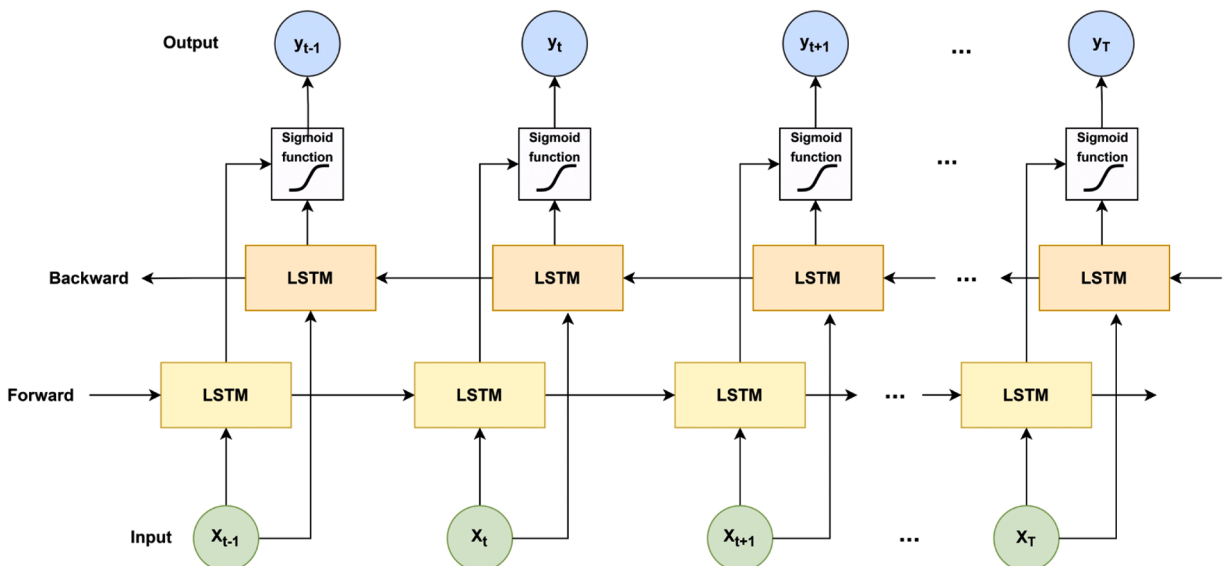


Fig. 4. Bidirectional LSTM architecture.

input, extracting feature maps, with the ReLU activation function commonly used (Eq. (13)). The pooling layer performs operations like max or average pooling, where max pooling identifies the most significant value within the filter's coverage area. In contrast, average pooling calculates the mean of the values in the area covered by the filter. This approach generates smaller outputs that still contain enough information for accurate decision-making. These layers are designed to extract essential features from the input data [18].

$$\text{ReLU}(x) = \max(0, x) \quad (13)$$

Before the fully connected layer, two- or three-dimensional data are flattened into a one-dimensional vector, which serves as input to the artificial neural network. Dropout can be applied in fully connected layers by randomly disabling neurons, thereby preventing the model from overfitting [26]. For binary classification, the sigmoid activation function (Eq. (5)) is used, while the softmax function (Eq. (6)) is preferred for multi-class classification. Finally, the model performs the classification and makes a prediction.

3.3.5. DNN

Deep Neural Net. consist of input, multiple hidden, and output layers, and represent an advanced form of artificial Neural Net. capable of extracting features from raw data. As shown in Fig. 6, the input layer receives raw data passed through hidden layers to extract high-level features [27]. The number of hidden layers determines the network's depth. DNNs use feedforward and backward propagation, processing input data through hidden layers to extract complex features [5]. Neuron connections involve multiplying weights by the preceding neuron's output, adding a bias (Eq. (14)), and applying activation functions like ReLU, sigmoid, or tanh. This enables the network to learn discriminative features from the input data. In Eqs. (14) and (15), k_j represents the total input of a neuron before entering the activation function. w_{ij} is the weight matrix, x_i is the input values, i.e. the outputs from the previous layer, and b_j is the bias value. σ is the activation function (ReLU, sigmoid, tanh, etc.) [1].

$$k_j = \sum_i (w_{ij} * x_i) + b_j \quad (14)$$

$$y_j = \sigma(k_j) \quad (15)$$

A loss function is employed to evaluate the network's predictions. In classification problems, cross-entropy is the preferred loss function (Eq. (16)). The backward derivative of the loss function is calculated and the weights and bias values are updated accordingly. This process is referred to as backpropagation [1]. In Eq. (16), y_i represents the actual label value, whereas $\hat{y}_i$ represents the model-predicted value.

$$L = -\sum_i y_i \log(\hat{y}_i) \quad (16)$$

Updating the weight and bias values in the deep neural network results in more accurate predictions. In the final layer of the network, one of two commonly used activation functions, the sigmoid or softmax, is selected, and the estimated values for the output variables are produced.

4. Proposed Method

4.1. Data preprocessing

To make the raw data meaningful and analysable, it is necessary to pre-process the data. This stage is essential for efficiency. URL data was initially taken as raw data and then pre-processed. The following steps were taken in the preprocessing phase.

- The column names in the data set were changed to 'url' and 'class'. Therefore, the resulting data set comprised n rows with two columns. In this context, n represents the number of URL addresses in the URL column. Four distinct data sets were employed in the study. Additionally, the number of URL addresses in each data set differs.

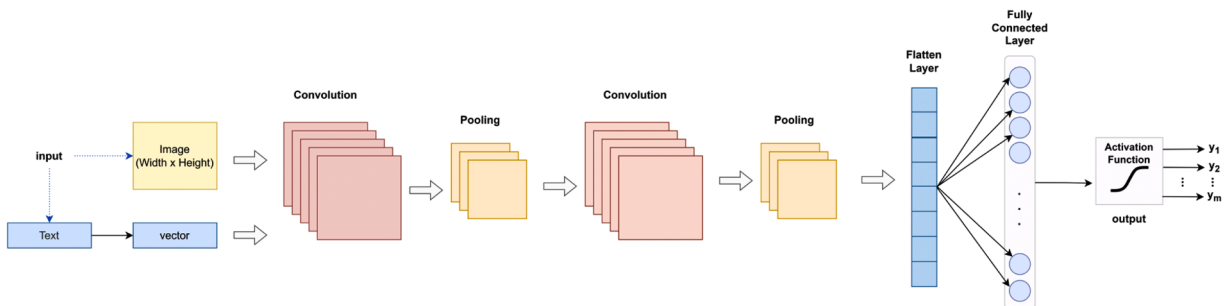


Fig. 5. CNN architecture.

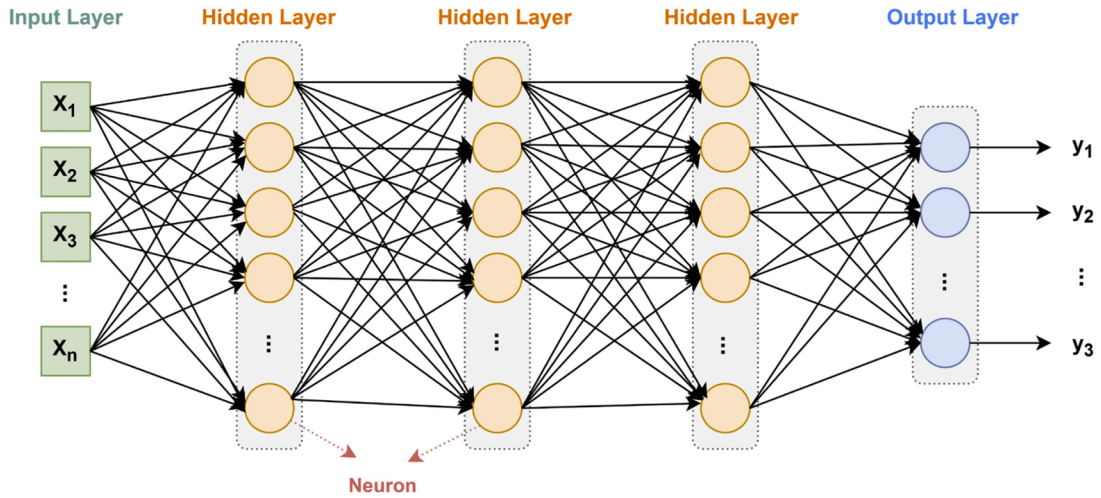


Fig. 6. DNN architecture.

- Unnecessary data were removed, and missing data were completed.
- The https:// and http:// components of the URLs were eliminated. All URLs in the data set include the 'https://' or 'http://' prefixes. As it is present in all URL addresses, it is not a distinctive element and has been removed.
- The 'class' column of the dataset shows the actual label values of the URL addresses. If the URL address has been categorically labelled as 'phishing', 'legitimate', 'malicious', or 'benign', label encoding was performed. In the label encoding process, the values

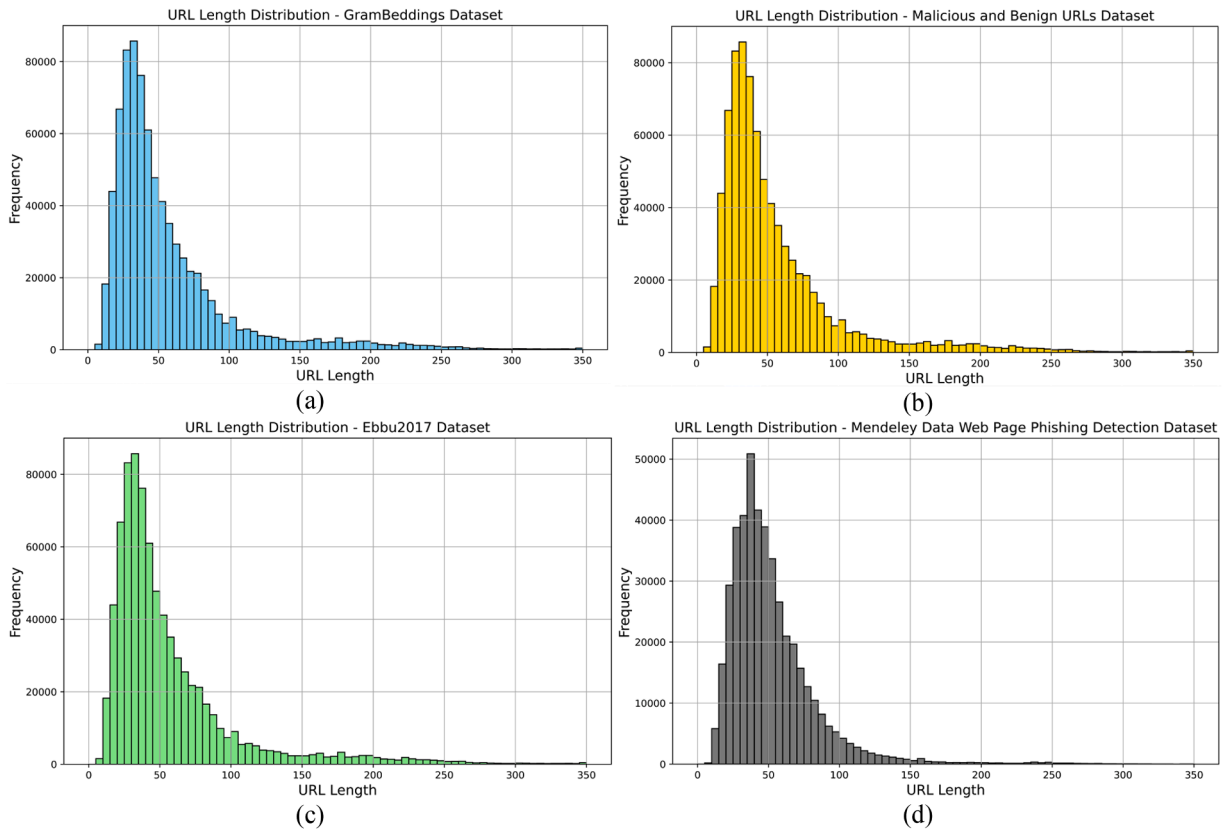


Fig. 7. (a) The distribution of the length of URLs in the GramBeddings dataset, (b) The distribution of the length of URLs in the Malicious and Benign URLs dataset, (c) The distribution of the length of URLs in Ebbu2017 Phishing dataset, (d) The distribution of the length of URLs in Mendeley Data Web Page Phishing Detection dataset.

assigned to the categories are as follows: 'phishing' is assigned a value of 1, 'legitimate' is assigned a value of 0, 'malicious' is assigned a value of 1, and 'benign' is assigned a value of 0. Label encoding aims to convert categorical data into a numerical form that algorithms can process.

- For each of the four data sets, the total number of URLs, the number of phishing URLs, the number of legitimate URLs, the maximum URL length, the minimum URL length, and the average URL length were calculated. The statistical results are presented in Table 1.

Fig. 7 illustrates the distribution of URL lengths in the GramBeddings, Malicious and Benign URLs, Ebbu2017 Phishing, and Mendeley Data Web Page Phishing Detection datasets. The graphs represent the frequency of different URL lengths, depicted as bars in the charts.

4.2. Feature extraction

The URLs were analysed as characters and words in the feature extraction phase. To achieve this, characterisation and tokenisation operations were performed.

4.2.1. Characterisation

The analysis of URL addresses was conducted on an individual character basis. As with label encoding, where categorical data is represented by numerical representation, each character is assigned a corresponding numerical value. Table 2 presents the values assigned to each character. This process yielded a character dictionary comprising character-value (char, value) pairs. The character dictionary consists of 150 characters. Standard American Standard Code for Information Interchange (ASCII) encoding was not utilised in constructing the character dictionary. ASCII employs a 7-bit encoding scheme to represent 128 distinct characters and covers the basic characters used in the English language. However, ASCII encoding was not preferred to eliminate language dependency. This approach allows for effectively processing URLs that contain characters not included in ASCII.

The maximum URL length was set at 250, 300, and 350 characters, and experiments were conducted to obtain results. For the sake of argument, let us assume that the maximum URL length is 300. In this case, the URL address is converted from text to numeric representation in the form of characters and corresponding values according to the character dictionary in Table 2. If the URL length is over 300 characters, the part after 300 characters is truncated and not converted to numeric representation. In the event that the URL length is less than 300 characters, the value 0 is added to complete the length to 300. The dictionary's character pair (PAD, 0) is employed in this instance. In the event that a character in the URL address is not defined in the dictionary, the UNK character is used, and the index of the encoded value corresponding to the UNK character is incremented by one for each unknown character. This is how the URL text is characterised and converted into numeric representations. The expressions are converted into numerical representations, referred to as character vectors. Character vectors are high-dimensional vectors comprising integers (Fig. 8). The methodology employed to generate the character vector is elucidated in Algorithm 1 between steps 22 and 30.

Fig. 8 illustrates a URL address in the GramBeddings database identified as a phishing attempt. After the preprocessing steps, the URL address is subsequently separated into its constituent characters. The value of max_url_length is set to 300. Each character is then assigned a value, and a character vector of length 300 is constructed. The resulting vector is of a high dimensionality. Before applying deep learning algorithms to high-dimensional character vectors, the embedding layer is utilised to generate low-dimensional, dense vector representations.

4.2.2. Character Embedding Layer

Fig. 10 illustrates the proposed model's architectural structure. This figure demonstrates the output shape of the data entering and leaving each layer. The embedding layer transforms high-dimensional character vectors into low-dimensional and dense vector representations. The shape of the vectors entering this layer as input data is (300). The shape of the vector exiting the character embedding layer is (300, 128). In this context, 128 represents the output size of the character embedding layer. The embedding layer is preferred to increase efficiency, create meaningful representations, and enable the model to process the data faster, thus obtaining results more quickly. In contrast to high-dimensional vectors, low-dimensional and dense vectors consume less memory, thereby reducing the training cost of the model. The use of dense vectors facilitates the processing and training of data. Consequently, the results are obtained more rapidly [14].

Table 2
Character dictionary representation.

Character (char)	Encode (value)
0 1 2 3 4 5 6 7 8 9	1–10
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z	11–36
a b c d e f g h i j k l m n o p q r s t u v w x y z	37–62
' ' ! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _ ` [1] ~ ¡ ¢ £ ¤ ¥ ¦ § ¨ © ª « ¬ ® ¯ ° ± ² ³ ´ µ ¶ · ¸ ¹ º » ¼ ½ ¾ À Á Â Ã Ä Å Æ × à á â ã ä å æ ç è é ê ë ì í î ï ð ñ ò ó ô õ ö ù ü ÿ ÿ — ‘ ’ , “ ” • … % ‰ € ™ \n	63–148
PAD	0
UNK	149

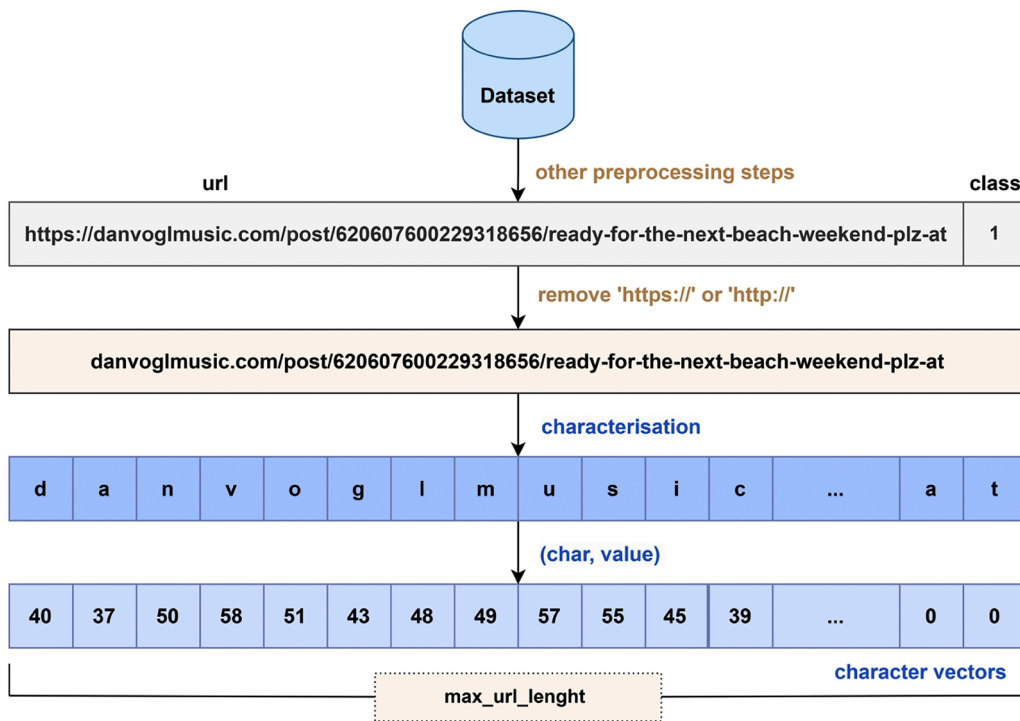


Fig. 8. Characterisation approach.

4.2.3. Tokenisation

The URL address is divided into tokens according to characters such as ‘/’, ‘.’, ‘-’, etc. Each token represents a word. Tokens can consist of meaningful or meaningless words, numbers, and characters. It’s important to highlight that the URL text is divided into tokens according to the characters ‘/’, ‘.’, ‘-’, etc. Table 3 presents the statistical information of the tokens obtained from the GramBeddings, Malicious and Benign URLs, and Ebbu217 Phishing databases. Upon analysis of Table 3, it is evident that the GramBeddings dataset has the highest number of tokens, totalling 5954,984 tokens. The more significant number of tokens in the GramBeddings data set can be attributed to the fact that the number of URL addresses in the data set is higher than in the other data sets. The number of tokens is calculated by dividing the URL address into the number of tokens that can be subdivided. In the GramBeddings dataset, the maximum number of tokens for a URL is 712. The maximum number of tokens is significant, as it determines the size of the vector when converting tokens to numeric representations. The minimum number of tokens indicates the minimum number of tokens a URL can be divided into. Table 3 indicates that the Malicious and Benign URLs dataset comprises 3469,951 tokens. The Ebbu2017 Phishing dataset has the lowest number of tokens, with 631,820 tokens, compared to other datasets. The maximum number of tokens for the Malicious and Benign URLs dataset is 166. The maximum number of tokens in the Ebbu 2017 Phishing dataset is 99. The minimum number of tokens in the GramBeddings and Malicious and Benign URLs datasets is 1. The minimum number of tokens in the Ebbu 2017 dataset is 2. The URL address of the Ebbu2017 Phishing dataset with the minimum number of tokens is ‘tvfplay.com/’. It should be noted that the ‘https://’ and ‘http://’ parts of the URL have been removed. Upon division of the URL into tokens according to the ‘.’ and ‘/’ characters, the tokens “tvfplay” and “com” can be obtained. This same methodology was employed in other datasets to ascertain the number of tokens. As illustrated in Table 3, the average number of tokens in the Ebbu2017 Phishing dataset is greater than that observed in the other datasets.

A distinct set of tokens was generated for each of the data sets. The numerical representation of the tokens is obtained through the Word2Vec artificial neural network model. The Word2Vec model is trained with the tokens. The Word2Vec model operates by the CBoW and skip-gram algorithms. In this study, the skip-gram algorithm is employed. The skip-gram algorithm does not ignore rare words that do not occur frequently in the text; it learns the context of rare words. Furthermore, it possesses greater information density

Table 3

The statistical analysis of tokens within data sets.

	GramBeddings Dataset [8]	Malicious and Benign URLs Dataset [22]	Ebbu2017 Phishing Dataset [2]
Total number of tokens	5.954.984	3.469.951	631.820
Maximum number of tokens	712	166	99
Minimum number of tokens	1	1	2
Average number of tokens	7.44	7.71	8.59

than CBoW, as it predicts the words situated to the left and right of a target word. A rare word in a URL address may possess the characteristics that render the URL address susceptible to phishing. Consequently, it may be beneficial to undertake a semantic analysis of rare words. The Word2Vec model employs a training process that utilises tokens, thereby enabling the creation of a word dictionary that reflects the semantic relationships between words. The dictionary comprises a list of words and their respective index values. (word, value) pairs are generated. The conversion of words in the URL text into vector representations is achieved through word and value pairs. The vectors are numerical representations that carry the meanings of the words. Each numerical value in the vector represents a feature. These vectors are referred to as word vectors and are high-dimensional vectors. The maximum number of tokens indicates the size of the word vectors (Fig. 9).

Table 4 presents the tokens obtained from the datasets and the word dictionaries generated by the Word2Vec model, which were learned separately for each dataset. The dictionary contains the word and the value corresponding to the word (word, value). The following results were obtained by analysing the word dictionary for each data set.

1. The words in the dictionaries may be meaningful words such as “login,” “authentication,” and “update”. Additionally, the dictionary may include meaningless words such as ‘slim,’ ‘yonks,’ and the number ‘927321’. As the URL is divided into tokens according to characters such as “/”, “.” and “-”, words consisting of a number or letter can be included as words in the dictionary. Given that the URL is divided into tokens, even single-character tokens are analysed as words.
2. Similar words may appear in dictionaries but do not necessarily carry the same value. To illustrate, the word ‘php’ is coded as 2 in the word dictionary of the GramBedding dataset, whereas it is coded as 3 in the Ebbu2017 dataset. The formation of this dictionary results from the semantic relationship and similarity between words being established according to the learning process in the Word2Vec model.
3. Since each dataset’s dimensions vary, the associated dictionaries’ size also differs. For instance, the dictionary of the GramBeddings dataset comprises 858,959 words, the dictionary of the Malicious and Benign URLs dataset contains 491,509 words, and the dictionary of the Ebbu2017 dataset consists of 68,263 words.
4. It should be noted that the total number of tokens in the data sets presented in Table 3 and the total number of words in the word dictionary provided in Table 4 do not correspond. It should be noted that a single word may be a token for more than one URL. The same word may appear in different URLs. The word dictionary is derived from the tokens, and a single entry in the dictionary represents each word.

4.2.4. Word Embedding Layer

The embedding layer transforms high-dimensional word vectors into low-dimensional and dense vector representations. The input data of the word embedding layer comprises word vectors. As illustrated in Fig. 10, the shape of the word vectors is (max_token_number). The output shape of the word embedding layer is (max_token_number, 128). The output shape of the vector from the word embedding layer is (712,128) for the GramBeddings dataset, (166,128) for the Malicious and Benign URLs dataset and (99,128) for the Ebbu2017 dataset. The word embedding layer differs from the character embedding layer in that the initial weight value is not randomised by default. The weight value is derived from the vector representations of words in the word dictionary. As the Word2Vec model has been trained, this weight value is pre-trained. This value is employed as the initial weight value in the word embedding layer. Given that no training exists in the character embedding layer, the initial weight value is randomly assigned as a default value. The pseudo-code of decomposing the URL into tokens and generating word vectors is presented in Algorithm 1 between steps 33 and

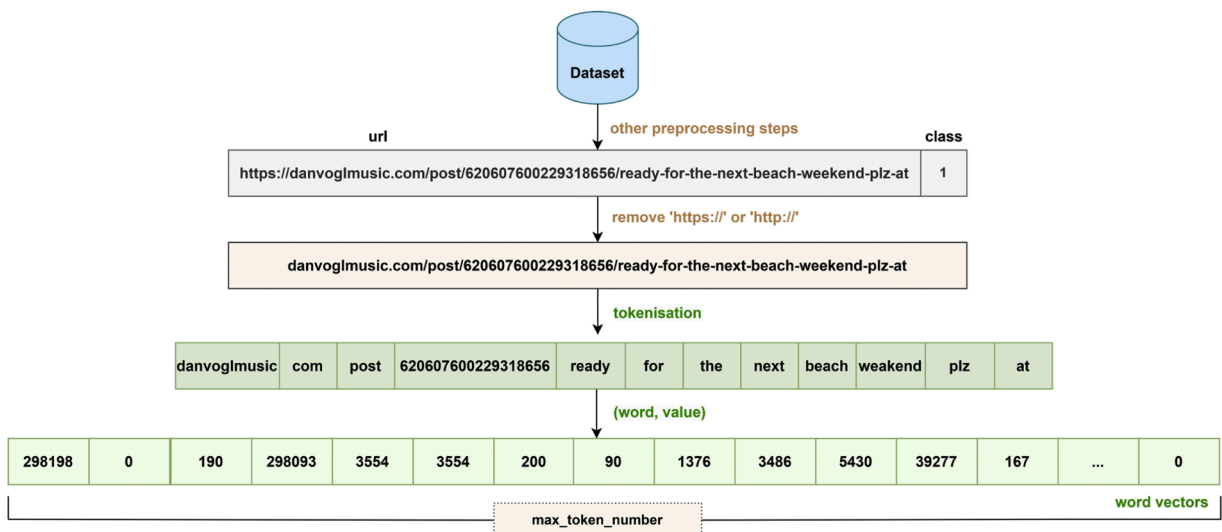


Fig. 9. Tokenisation approach.

Table 4

Word dictionary representation for datasets.

Word Dictionaries					
GramBeddings Dataset [8]		Malicious and Benign URLs Dataset [22]		Ebbu2017 Phishing Dataset [2]	
Token (word)	Encode (value)	Token (word)	Encode (value)	Token (word)	Encode (value)
com	0	www	0	com	0
www	1	com	1	www	1
php	2	html	2	amp	2
html	3	php	3	php	3
login	4	org	4	html	4
...	...	...	...	...	...
verification	71	n	71	update	71
...	...	...	...	...	...
927,321	565	charles	565	interactive	565
...	...	...	...	...	...
authentification	754	thomas	754	fi	754
...	...	...	...	...	...
rlzm	858,959	yonks	491,509	jesuscreed	68,263

51.

4.3. Proposed Architecture

The pseudo-code of the model proposed in this study is presented in Algorithm 1. The architectural structure of the model proposed in Algorithm 1 is depicted in Fig 10.

Algorithm 1 A proposed deep learning model algorithm for the classification of URLs

Input: URL text data

Output: Phishing or Legitimate

1: Loading of the data set.

2: Unnecessary data is removed from the data set.

3: Missing data is completed.

4: The data set consists of two columns: URL and class.

5: The categorical data in the 'class' column has been recorded.

6: 0 is assigned to legitimate URLs. (legitimate = 0)

7: 1 is assigned to phishing URLs. (phishing = 1)

8: Separating URLs and class labels.

9: $X = \text{dataset}['\text{url}']$

10: $Y = \text{dataset}['\text{class}']$

11: $\text{original_urls} = X$

12: **for** url in $\text{original_urls}(\text{len}(\text{url}))$ **do**

13: $\text{original_url_lengths} = []$ % A list of the lengths of URLs is generated.

14: **end for**

15: $\text{max_url} = \max(\text{original_urls}, \text{key}=\text{len})$

16: $\text{min_url} = \min(\text{original_urls}, \text{key}=\text{len})$

17: $\text{total_urls} = \text{len}(\text{original_urls})$

18: $\text{average_length} = \text{sum}(\text{original_url_lengths}) / \text{total_urls}$

19: $\text{cleaned_urls} = []$

20: https:// and http:// parts of URLs are removed.

21: $\text{max_url_length} = 300$

22: Identifying the characters of the character dictionary

23: Create a dictionary of characters (char_dict)

24: **for** index, char in $\text{enumerate}(\text{allowed_characters})$ **do**

25: add the pair (character, index + 1) to char_dict

26: $\text{char} : \text{index} + 1$

27: **end for**

28: $\text{char_dict}['\text{<PAD>}'] = 0$ % Add 0 for the padding character.

29: $\text{char_dict}['\text{<UNK>}'] = \text{len}(\text{char_dict})$ % Add a new index for unknown character.

30: Convert url to character vectors

31: Separation of the data set as 80% training ($X_{\text{train_char}}$) and 20% test ($X_{\text{test_char}}$)

32: Separation of the training dataset into 80% training and 20% validation

33: Split urls into tokens (tokenize_url)

34: $\text{dataset}['\text{tokens}'] = \text{dataset}['\text{url}']. \text{apply}(\text{tokenize_url})$

35: Creating Word2Vec model for tokens (sentences, vector_size, window, min_count, workers, sg)

36: Creating a word dictionary ((key, index) pairs) % key=word, index=value (word, value) pairs

37: Creating word embedding matrix

38: **for** word, i in $\text{word2vec_model.wv.key_to_index.items}()$ **do**

(continued on next page)

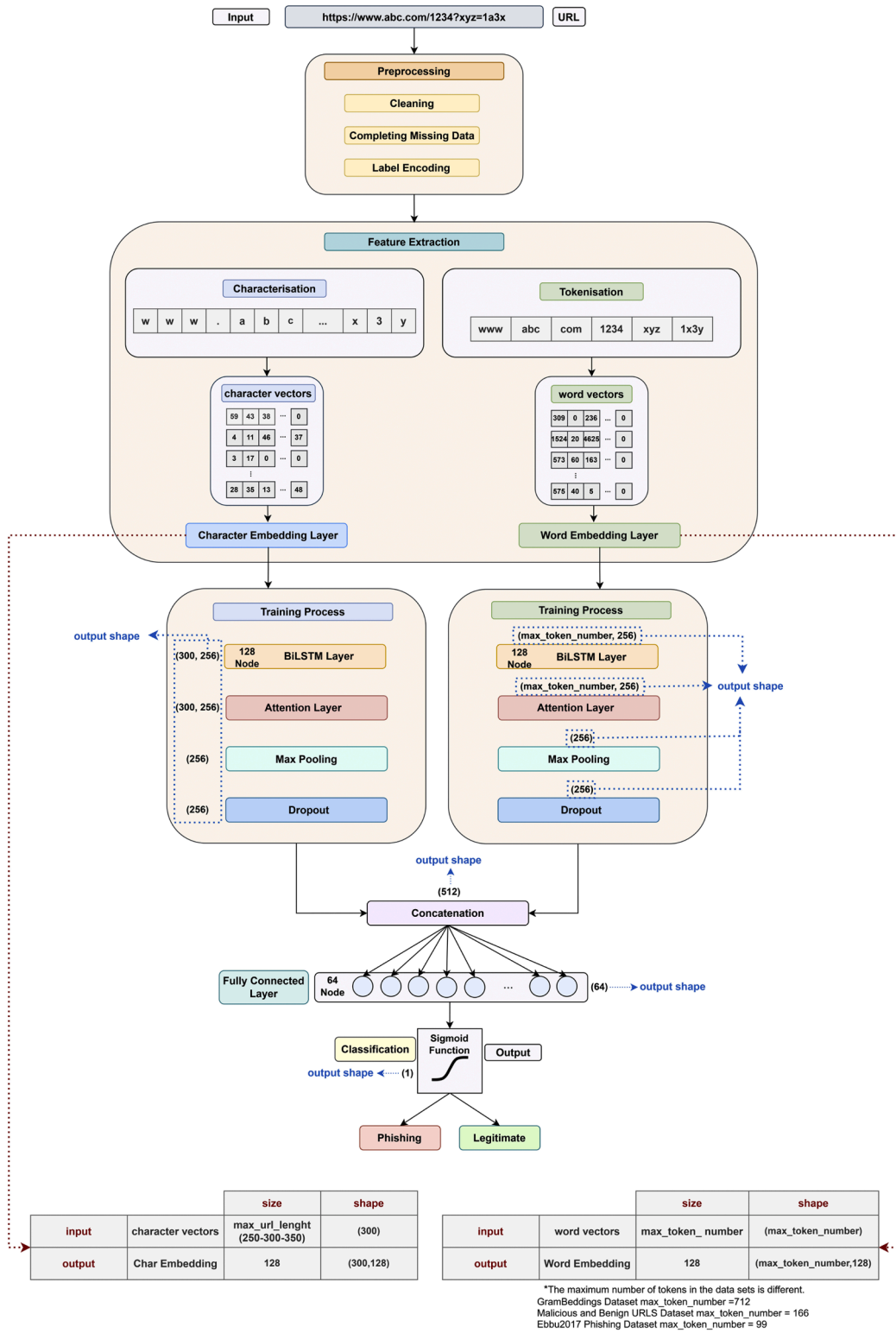


Fig. 10. Proposed model architecture.

(continued)

```

39: embedding_matrix[i] = word2vec_model.wv[word] % embedding_matrix is used as a pre-trained weight in the word embedding layer
40: end for
41: token_lists = dataset['tokens']
42: for tokens in token_lists do
43: token_number = len(tokens)
44: end for
45: total_tokens = sum(token_number)
46: min_token_number = min(token_number) % The smallest number of tokens in the tokenised versions of all URLs is represented.
47: max_token_number = max(token_number) % The longest number of tokens in the tokenised versions of all URLs is represented.
48: average_token_number = total_tokens / len(token_lists)
49: Separation of the data set as 80% training (X_train_word) and 20% test (X_test_word)
50: Separation of the training dataset into 80% training and 20% validation
51: Convert to tokenised URLs to numeric arrays (word_vectors)
52: Create a model
53: An input layer (character_input) is created to receive the character sequence. % This approach provides a character-by-character representation of URLs.
54: The character input is passed to an embedding layer containing a vector representation of the characters. % The char_dict dictionary represents the indices of the characters.
55: The output of the embedding layer is passed to a Bidirectional LSTM layer. % This layer utilises both forward and backward sequential information.
56: The output of the BiLSTM is forwarded to an attention layer. % This allows for the focus on specific parts of the BiLSTM output.
57: The output of the attention layer is forwarded to a global maximum pooling layer (GlobalMaxPooling1D). % This process reduces the size of the features by taking the most significant values of the output of the attention layer.
58: A dropout layer (Dropout) is added to the model to prevent overfitting.
59: An input layer (word_input) is created to receive the word sequence. % This approach permits the representation of URLs on a word basis.
60: The word input is passed to an embedding layer using a pre-trained word embedding matrix. % This matrix contains the word vectors of the Word2Vector model and cannot be trained.
61: The output of the embedding layer is transmitted to a Bidirectional LSTM layer. % This layer utilises both forward and backward sequential information.

```

(continued on next page)

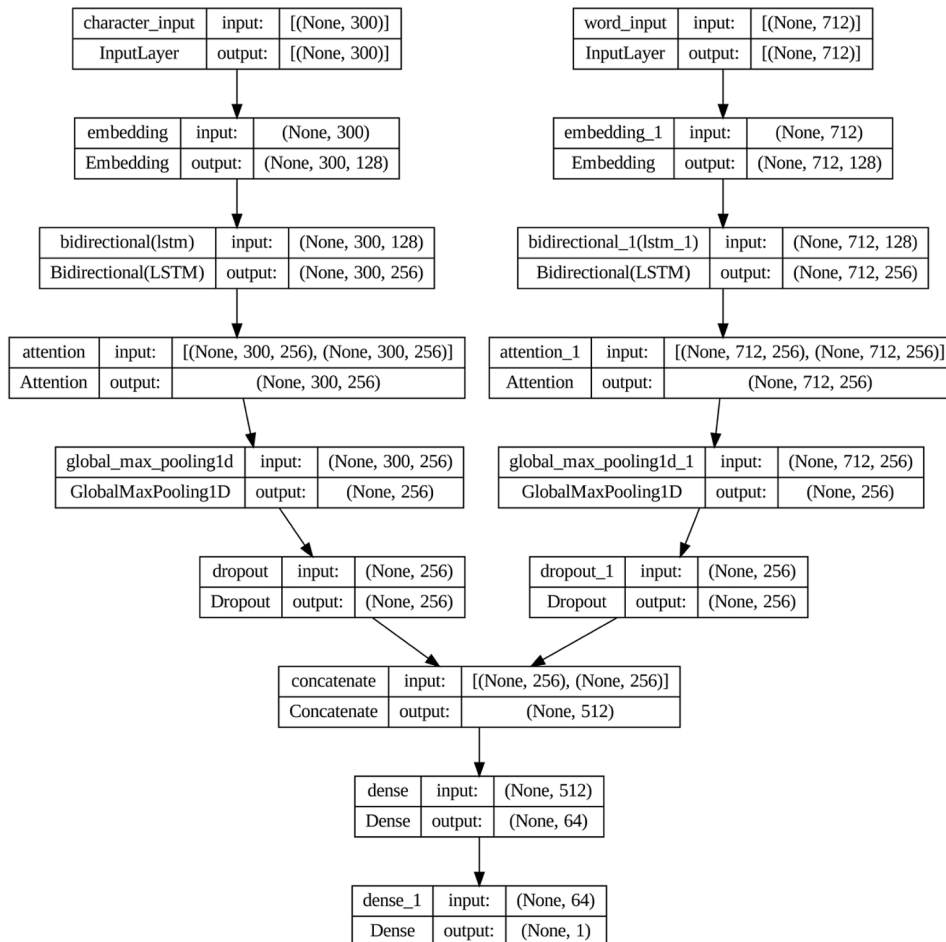


Fig. 11. Proposed model layers in GramBeddings Dataset.

(continued)

```

62: The output of the BiLSTM is transmitted to an attention layer.
63: The output of the attention layer is forwarded to a global maximum pooling layer (GlobalMaxPooling1D).
64: A dropout layer (Dropout) is added to prevent overfitting.
65: The character and word-based feature outputs are combined into a single vector.
66: A fully connected layer is established, employing the ReLU activation function.
67: An output layer with a sigmoid activation function is added.
68: The model is compiled with the Adam optimisation algorithm and the binary cross entropy loss function.
69: for epoch in range (30) do
70: Evaluate Accuracy and Validation Accuracy
71: Evaluate Loss and Validation Loss
72: Evaluate Confusion Matrix
73: Evaluate Precision, Recall, F1-Score, ROC Curve, AUC
74: end for

```

4.3.1. BiLSTM Layer

BiLSTM represents an advanced model of LSTM that employs a bidirectional feedforward and backward mechanism. The model processes sequential data and learns the sequential links within the data. The algorithm is suitable for use with URL data. As it is capable of operating in both directions, the model can extract more information from the data [28]. Fig. 10 illustrates the input of the character embedding layer to the BiLSTM model. The BiLSTM model comprises 128 neurons. As it operates in both directions, the output shape becomes 256. The output of the BiLSTM layer is then provided as input to the attention layer, with the shape (300, 256). The value 300 represents the maximum length of a URL. The same process is then applied to the data from the word embedding layer. The BiLSTM algorithm with 128 neurons is also employed to extract word-based features. The output shape is (max_token_number, 256). As illustrated in Fig. 11, given that the maximum token number of the GramBeddings dataset is 712, the output shape is (712, 256).

4.3.2. Attention Layer

The Attention layer employs a weighting system, whereby different inputs are assigned differing weights according to the logic governing its functioning. The attention weight is calculated to ensure that specific inputs are given priority. Consequently, the model prioritises the processing of crucial information. Furthermore, the attention layer enhances the performance of the model by preventing the loss of information in long sequences [29]. The operational principle is as follows: The process's three fundamental elements are the query, key, and value. In this context, the term "query" represents the word or character to be considered. The term "key" is used to describe all inputs, while the term "value" is used to represent the actual values of all inputs. Firstly, the resemblance between the query and key inputs is calculated. The similarities are normalised using the softmax function, and weights are obtained. The weights obtained are then multiplied by all values, which are summed. This process yields the output of the attention layer. Consequently, the inputs (query, key, value) are processed and transformed into the attention vector [29].

In Eq. (17), the symbols q , k , and v represent, respectively, the query, the key, and the value. The symbol d_k represents the size of the key vectors. The operation performed between q and k is the dot product. The symbol A is used to represent the attention weight sum.

$$A(q, k, v) = \text{softmax}\left(\frac{q \cdot k^T}{\sqrt{d_k}}\right) * v \quad (17)$$

The output shape of the Attention layer is (300, 256) for character-based input and (max_token_number, 256) for word-based input. As the Grambedding dataset is employed in Fig. 10, the output shape is (712, 256).

4.3.3. Max Pooling

The max pooling layer takes the maximum values in the attributes from the previous layer and highlights the prioritised information. The size of the input data is not the same as that of the output data. As can be seen in Fig. 10 and Fig. 11, the output shape of the data entering the max pooling layer is (256). The computational cost is reduced, and the model runs faster as the size is reduced.

4.3.4. Dropout

During the dropout process, a random selection of neurons is deactivated. No connection is made with the deactivated neurons. Dropout is used to prevent the model from overfitting. The dropout value was randomly selected as 0.2. As illustrated in Fig. 10 and Fig. 11, the input shape of the data provided as input to the Dropout layer is (256). The output shape of the data is also (256).

4.3.5. Concatenation and Classification

Training is performed separately for character and word-based features. Character vectors are high-dimensional vectors containing character-based features. High-dimensional vectors are transformed into low and dense representational vectors in the character embedding layer, which are then available for classification in deep learning algorithms. Training occurs within layers, including BiLSTM, attention, max pooling, and dropout. Each layer performs a different process. The exact process is then applied to word-based features. The word embedding layer converts high-dimensional word vectors into low and dense vector representations. Training is

then performed on word-based features in the BiLSTM, such as attention, max pooling, and dropout layers. The outputs of the character and word-based models obtained after training are combined in the concatenate layer. This provides a more comprehensive, powerful, and richer representation. In the concatenate layer, character-based and word-based features are combined, resulting in an output shape of (512). A fully connected layer processes the concatenated features and derives higher-level representations. In the fully connected layer, the number of neurons is set to 64, and the shape of the input data (512) is transformed into the output shape (64) through this layer. The ReLU activation function (Eq. (13)) is used in this layer. Finally, since binary classification is performed in the output layer, the sigmoid activation function (Eq. (5)) is employed, and classification is performed. The classification result can be either phishing (1) or legitimate (0). All these processes are explained between steps 52 and 68 in Algorithm 1. Fig. 10 visually illustrates the proposed model. Fig. 11 illustrates the layers generated by the proposed model when applied to the GramEmbedding dataset. The figure shows the respective input and output shapes of each layer.

4.4. Train-Validation-Test Split

The application of k-fold cross-validation led to an overload in Random Access Memory (RAM) during training, mainly due to the large size of the GramBeddings dataset. Since this method divides the dataset into k parts and uses each part repeatedly for training and validation, it requires large chunks of data for each fold to be kept in RAM at the same time. This resulted in insufficient RAM and performance problems. Therefore, the datasets were initially randomly divided into two distinct subsets in an 80–20 ratio, one designated for training and the other for testing. Subsequently, the training dataset underwent another random split into training and validation subsets, also in an 80–20 ratio (Fig. 12). The rationale behind the separation of the dataset into three distinct categories (train, validation, and test) is to enable the accurate evaluation of machine and deep learning models, as well as to enhance their generalisation capabilities. Experimental studies were conducted on the Google Colab utilising the NVIDIA Tesla T4 GPU. All datasets were trained and tested in this environment. The experiments were performed on cloud-based Google servers with high RAM. The server environment comprised 51 GB of system RAM, 15 GB of GPU and 201.2 GB of disc memory. The Python 3 programming language and the Keras and TensorFlow libraries were employed. Version 2.15 of the Keras and TensorFlow libraries was preferred. The pseudo-code for the train-validation-test split is presented in Algorithm 1, steps 31 and 32 for character-based features and steps 49 and 50 for word-based features.

5. Experimental Results

This section of the paper presents the step-by-step development process of the proposed model, which is given in Fig. 10, and discusses the experimental findings in a cause-and-effect relationship. The experimental findings section employs a question-and-answer format to illustrate the development process of the proposed model. To develop the proposed model and obtain the optimum results that the model can achieve, a series of questions were posed, experiments were conducted, and the model's development process was followed by the answers received.

5.1. Classification metrics

Given that determining whether a URL address indicates a phishing attempt or a legitimate site is inherently a classification problem, the system's performance is evaluated using classification metrics. In the classification process, a confusion matrix is employed to ascertain the accuracy of the data categorisation. In accordance with the confusion matrix, statements for which the predicted and actual values are positive are classified as true positives (TP). Conversely, statements in which the predicted value is negative but the actual value is positive are designated as false negatives (FN). Similarly, statements for which the predicted value is positive but the actual value is negative are identified as false positives (FP). Finally, statements with negative predicted and actual values are classified as true negatives (TN). The expressions mentioned above are employed in calculating various statistical metrics,

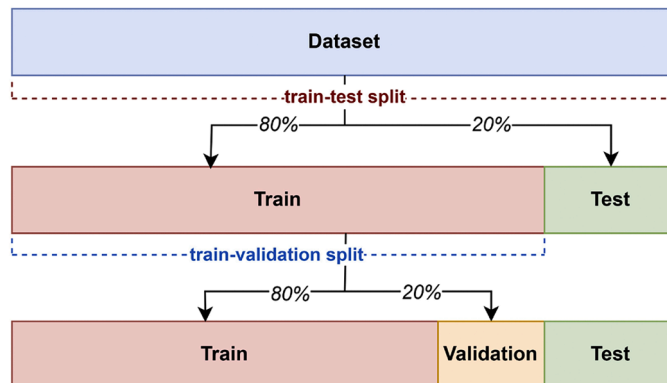


Fig. 12. Separation of datasets into the train, validation and test.

including accuracy, precision, recall, F1-score, and Area Under Curve (AUC). The concept of ‘accuracy’ is denoted by the proportion of correct predictions made by a model out of all predictions. This is calculated utilising the formula outlined in Eq. (18).

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (18)$$

Precision is defined as the proportion of accurately predicted positive instances relative to all samples predicted by the model. This metric, calculated using the formula in Eq. (19), indicates the accuracy of the model’s positive forecasts.

$$Precision = \frac{TP}{TP + FP} \quad (19)$$

The recall metric represents the proportion of samples correctly identified as positive by the model. The recall metric is calculated using the formula provided in Eq. (20). In essence, it is an indicator of the model’s capacity to identify positive instances accurately.

$$Recall = \frac{TP}{TP + FN} \quad (20)$$

The F1-score is a statistical metric that takes the harmonic mean of precision and recall values and reduces them to a single number. The F1-score is employed to identify the optimal model from a set of models exhibiting disparate precision and accuracy values. The F1-score is a value that ranges from 0 to 1. 1 represents the highest achievable value, while 0 represents the lowest. Eq. (21) illustrates the formula used to calculate the F1-score.

$$F1 - Score = 2 * \left(\frac{Precision * Recall}{Precision + Recall} \right) \quad (21)$$

True Positive Rate (TPR) is a metric that indicates the proportion of real phishing URLs that the model correctly predicts. A high TPR value indicates that the model has a high ability to identify phishing URLs. This metric is calculated using Eq. (22). The recall value is the same as the TPR.

$$TPR = \frac{TP}{TP + FN} \quad (22)$$

The True Negative Rate (TNR) represents the proportion of the actual legitimate URLs that the model correctly predicts. It is calculated using Eq. (23). A high TNR indicates the model’s capacity to identify legitimate URLs accurately.

$$TNR = \frac{TN}{TN + FP} \quad (23)$$

The False Positive Rate (FPR) represents the proportion of legitimate URLs misclassified as phishing by the model, as determined by Eq. (24). A decreased FPR indicates that the model accurately distinguishes legitimate URLs.

$$FPR = \frac{FP}{FP + TN} \quad (24)$$

The False Negative Rate (FNR) represents the percentage of phishing URLs that the model has mistakenly identified as legitimate. This is calculated via Eq. (25). A reduced FNR indicates that the model accurately categorises phishing URLs.

$$FNR = \frac{FN}{FN + TP} \quad (25)$$

The Receiver Operating Characteristic (ROC) curve is a graphical representation of the correlation between the FPR on the x-axis and the TPR on the y-axis. The performance of the model is evaluated by the proximity of the ROC curve to the y-axis. A model with a more elevated and centralised ROC curve is considered superior. The AUC encapsulates the total area beneath the ROC curve, concisely summarising the model’s overall performance. This metric highlights the model’s capacity to differentiate between phishing and legitimate URLs, with values ranging from 0 to 1. A value approaching 1 indicates the model’s efficacy in distinguishing between the two categories [1]. In conclusion, a ROC curve characterised by a high TPR, a low FPR and a substantial AUC value demonstrates the model’s ability to identify phishing URLs while minimising false positives accurately.

5.2. Experiments

In this section, the experiments performed and the findings obtained are presented. The development of the model is based on detailed experimental results.

5.2.1. Selection of the best optimisation algorithm

Given that the classification of URL addresses as phishing or legitimate is a binary classification problem, the binary cross entropy loss function is employed in this study. The loss function enhances the efficacy of the deep learning model by quantifying the discrepancy between the actual and predicted classes. Optimisation algorithms are employed to minimise the loss function utilised while training deep learning models, enhancing the model’s classification performance. The Stochastic Gradient Descent (SGD),

Adaptive Moment Estimation (Adam) and Root Mean Square Propagation (RMSprop) algorithms are commonly used optimisation algorithms [14]. In this study, the SGD, Adam and RMSprop algorithms were trained and tested on the Malicious and Benign URLs and Ebbu2017 Phishing datasets to determine the most suitable optimisation algorithm for the proposed model. The Malicious and Benign URLs and Ebbu2017 datasets were selected for analysis due to their relatively smaller size. The LSTM algorithm was employed as the classification algorithm, with training conducted over 10 epochs at a learning rate of 0.01.

The rationale behind the selection of the LSTM algorithm in the initial experiment was its prevalence in the literature as a means of capturing semantic and sequential dependency features in character sequences. The test accuracy rates obtained from the experiment were subjected to a comparative analysis. As illustrated in Table 5, the Adam algorithm demonstrated superior performance to other algorithms, with an accuracy value of 0.9656 in the Malicious and Benign URLs Dataset and 0.9405 in the Ebbu2017 Phishing Dataset. Furthermore, Wang et al. [10], Afzal et al. [11], and Ozcan et al. [1] demonstrated successful classification performance with the Adam optimisation algorithm in their respective studies. Based on the findings presented in Table 5 and those of analogous studies in the literature, the Adam algorithm was selected for use in this study.

5.2.2. Algorithm selection for character-based features

In the literature, various algorithms have been used for phishing attack classification. Rao et al. [17] employed LSTM, Yang et al. [14] combined LSTM and CNN, Roy et al. [19] used LSTM and BiLSTM, and Ozcan et al. [1] applied LSTM and DNN. This study conducted experiments with LSTM, BiLSTM, CNN, and DNN to identify the most suitable algorithm for training character-based features and developing the proposed model. The training, validation, and test results are summarised in Table 6.

In order to ascertain the most suitable algorithm for training character-based features, training and testing were conducted on the Malicious and Benign URLs and Ebbu2017 Phishing datasets. The two datasets were initially selected for analysis because they are smaller than the GramBedding dataset, and one has a balanced and the other an unbalanced data distribution. Furthermore, experiments were conducted with the GramBeddings dataset during the subsequent stages of model development. Upon analysis of the results presented in Table 6, it can be observed that the train, validation, and test accuracy values of the BiLSTM algorithm are higher than those of the other algorithms. As the BiLSTM algorithm operates in a bidirectional manner, the training time was longer than that of the other algorithms, yet the results were more successful in performance. Consequently, the BiLSTM algorithm was employed to train character-based features.

5.2.3. Algorithm selection for word-based features

This section presents the findings of a series of experiments to identify the optimal algorithm for training word-based features. The Word2Vec model was employed to transform the tokens derived from URL addresses into vector representations. Experiments were conducted with skip-gram and CBoW algorithms to ascertain which algorithm should be used in the Word2Vec model to create vector representations of words. Table 7 presents the outcomes of the skip-gram algorithm, while Table 8 displays the outcomes of the CBoW algorithm. The tokens obtained from the URL address were converted into vector representations, and the training, validation, and testing processes of word-based features were performed with LSTM, BiLSTM, CNN, and DNN algorithms.

Upon examination of the data presented in Table 7 and Table 8, the following observations can be made:

- The algorithm most suitable for training word-based features is the BiLSTM algorithm.
- Following the training and testing of the skip-gram algorithm in the Word2Vec model, the test accuracy was found to be 0.9918 in the Malicious and Benign URLs dataset and 0.9761 in the Ebbu2017 Phishing dataset.
- Following training and testing using the CBoW algorithm in the Word2Vec model, an accuracy of 0.9866 was achieved for the Malicious and Benign URLs dataset, while an accuracy of 0.9827 was achieved for the Ebbu2017 Phishing dataset.
- The skip-gram algorithm demonstrated higher success rates in the Malicious and Benign URLs dataset.
- In the Ebbu2017 Phishing dataset, the CBoW algorithm demonstrated a higher success rate.
- The results of the 10-epoch training demonstrate that the duration of training using the skip-gram and BiLSTM algorithms is less than that of training using the CBoW and BiLSTM algorithms.

The size of the Malicious and Benign URLs dataset is greater than that of the Ebbu2017 Phishing dataset. The skip-gram algorithm demonstrated superior performance to the CBoW algorithm on the Malicious and Benign URLs dataset. The GramBeddings dataset is also a large one. Considering the abovementioned factors, selecting the skip-gram algorithm for this study is more appropriate. The CBoW algorithm is trained with high-frequency words more frequently found in the dictionary. Therefore, it performs better on high-

Table 5

The selection of the optimisation algorithm to be employed in the model.

Dataset	Optimisation Algorithm	Test Accuracy	Learning Rate	Epoch	Classification Algorithm
Malicious and Benign URLs Dataset [22]	SGD	0.7695	0.01	10	LSTM
	Adam	0.9656	0.01	10	LSTM
	RMSprop	0.7680	0.01	10	LSTM
Ebbu2017 Phishing Dataset [2]	SGD	0.5053	0.01	10	LSTM
	Adam	0.9405	0.01	10	LSTM
	RMSprop	0.4967	0.01	10	LSTM

Table 6

The selection of an algorithm for the training of character-based features.

Dataset	Algorithm	Training Accuracy	Validation Accuracy	Test Accuracy	Training Time (sec.)	Epoch
Malicious and Benign URLs Dataset [22]	LSTM	0.9888	0.9869	0.9869	1286.10	10
	BiLSTM	0.9901	0.9886	0.9881	2246.09	10
	CNN	0.9740	0.9722	0.9716	564.50	10
	DNN	0.9660	0.9645	0.9647	435.51	10
Ebbu2017 Phishing Dataset [2]	LSTM	0.9354	0.9283	0.9308	326.42	10
	BiLSTM	0.9493	0.9491	0.9474	455.35	10
	CNN	0.8548	0.8361	0.8270	143.08	10
	DNN	0.7443	0.7441	0.7460	83.69	10

Table 7

The selection of an algorithm for training word-based features using the skip-gram algorithm in the Word2Vec model.

Dataset	Algorithm	Training Accuracy	Validation Accuracy	Test Accuracy	Training Time (sec.)	Epoch	Word2Vec Algorithm
Malicious and Benign URLs Dataset [22]	LSTM	0.7700	0.7704	0.7698	1346.56	10	Skip-gram
	BiLSTM	0.9999	0.9914	0.9918	1839.94	10	Skip-gram
	CNN	0.7679	0.7685	0.7680	508.22	10	Skip-gram
	DNN	0.7679	0.7685	0.7680	442.87	10	Skip-gram
Ebbu2017 Phishing Dataset [2]	LSTM	0.5042	0.5041	0.5053	204.37	10	Skip-gram
	BiLSTM	0.9998	0.9765	0.9761	220.99	10	Skip-gram
	CNN	0.7391	0.7640	0.7672	83.32	10	Skip-gram
	DNN	0.5050	0.5041	0.5053	63.21	10	Skip-gram

Table 8

The selection of an algorithm for training word-based features using the CBoW algorithm in the Word2Vec model.

Dataset	Algorithm	Training Accuracy	Validation Accuracy	Test Accuracy	Training Time (sec.)	Epoch	Word2Vec Algorithm
Malicious and Benign URLs Dataset [22]	LSTM	0.7716	0.7735	0.7728	1644.74	10	CBoW
	BiLSTM	0.9999	0.9863	0.9866	2287.87	10	CBoW
	CNN	0.7679	0.7685	0.7680	571.81	10	CBoW
	DNN	0.7679	0.7685	0.7680	432.48	10	CBoW
Ebbu2017 Phishing Dataset [2]	LSTM	0.5030	0.5041	0.5053	171.19	10	CBoW
	BiLSTM	0.9999	0.9795	0.9827	264.97	10	CBoW
	CNN	0.7437	0.7654	0.7598	87.46	10	CBoW
	DNN	0.5056	0.5041	0.5053	82.69	10	CBoW

frequency words. In contrast, the skip-gram algorithm is trained with low-frequency words that rarely occur in the dictionary. Consequently, it performs better on low-frequency words than CBoW.

A word dictionary was created with the tokens obtained from the URL addresses in the data set (Table 4). Each word in this dictionary can be found in different URLs in the data set. A word can appear in both the URL address labelled as phishing and the URL address labelled as legitimate. Therefore, low-frequency cases where words occur rarely are essential. This is because a low-frequency word can be the distinguishing feature of a phishing URL address. The findings and conclusions presented in Table 7 and Table 8 show that the skip-gram algorithm is an appropriate choice for generating vector representations, while the BiLSTM algorithm is well-suited for training word-based features in the proposed model.

Table 9

The selection of an appropriate URL length by the data sets.

Dataset	URL Length	Training Accuracy	Validation Accuracy	Test Accuracy	Training Time (sec.)	Epoch
GramBeddings Dataset [8]	250	0.9949	0.9818	0.9820	44,232.77	30
	300	0.9951	0.9815	0.9824	45,100.84	30
	350	0.9926	0.9804	0.9809	47,221.18	30
Malicious and Benign URLs Dataset [22]	250	0.9985	0.9927	0.9937	10,307.31	30
	300	0.9985	0.9935	0.9932	10,830.34	30
	350	0.9981	0.9930	0.9927	12,299.67	30
Ebbu2017 Phishing Dataset [2]	250	0.9984	0.9850	0.9836	1426.76	30
	300	0.9986	0.9845	0.9834	1523.78	30
	350	0.9973	0.9838	0.9846	1766.47	30

5.2.4. Selection of the optimal URL length

At the outset of the training process for word-based features, the value of *max_token_number* is determined according to the dataset. The maximum number of tokens is 712 for the GramBeddings dataset, 166 for the Malicious and Benign URLs dataset, and 99 for the Ebbu2017 Phishing dataset. Three values were selected for *max_url_length*: 250, 300, and 350 to determine the optimal URL length for character-based attributes. Subsequently, experimental observations were made. Section 4, Fig. 7 illustrates the distribution of URL lengths according to the datasets. Based on the distributions observed, 250, 300, and 350 values were selected. Table 9 presents the outcomes of the model presented in Fig. 10 and Algorithm 1 for all three datasets following the training, validation, and testing processes for 250, 300, and 350 URL lengths. The results indicated that the test accuracy was 0.9824 at 300 URL lengths in the GramBeddings dataset, 0.9937 at 250 URL lengths in the Malicious and Benign URLs dataset, and 0.9846 at 350 URL lengths in the Ebbu2017 Phishing dataset.

Fig. 7 (a) illustrates that most URL length distributions for the GramBedding dataset are below 300. Upon examination of Fig. 7 (b), it becomes evident that the majority of URLs in the Malicious and Benign URLs dataset exhibit a length distribution below 250–270 values. Fig. 7 (c) reveals that the Ebbu2017 Phishing dataset exhibits a similar trend, with most URLs exhibiting a length below 300–320 values. The results presented in Table 9 and Fig. 7 are consistent.

It is recommended that an average value be taken for the proposed model. To achieve this, the value 300 was selected as the maximum URL length among the 250, 300, and 350 values to be utilised in the training process of character-based features. Selecting 250 results in excluding specific URLs from the GramBeddings and Ebbu2017 Phishing datasets. Selecting a value of 350 increases the time required for training the Grambeddings and Malicious URLs datasets. It can be observed that the optimal value for the proposed model is 300.

5.2.5. Selection of the number of BiLSTM neurons

To assess the efficacy of the proposed model across a range of neuron counts, we conducted experiments with neurons 32, 64, 128, and 256 within the BiLSTM network. The training process was conducted over 20 epochs. Table 10 presents the results for all three data sets. Table 10 indicates that the BiLSTM network with 64 and 128 neurons achieved a classification success of 0.9819 on the GramBeddings dataset with the test dataset. Furthermore, the training, validation, and test accuracy performance of the 128-neuron BiLSTM network was superior to that of other neuron numbers.

Upon examination of the results in the Malicious and Benign URLs dataset (Table 10), it becomes evident that the test accuracy success observed in the BiLSTM network with 32, 64, and 128 neurons is identical at 0.9933. Additionally, it can be seen that the training, validation, and test results obtained with 128 neurons are generally better than those obtained with other neuron numbers in the Malicious and Benign URLs dataset. An analysis of the Ebbu2017 Phishing dataset, as shown in Table 10, reveals that the BiLSTM network with 256 neurons exhibits higher training accuracy. However, the validation and test values are higher in the network with 128 neurons, with 0.9834 and 0.9831, respectively.

Table 10 is analysed for all data sets. The classification performance and generalisation capability of the 128-neuron BiLSTM network is better. The training and testing times of the data sets are also given in Table 10. It can be observed that the training and testing times increase as the number of neurons in the BiLSTM network increases. Test data are data the model has never seen during training. In real-world scenarios, response time is significant in phishing detection, as it determines whether a URL address encountered for the first time is legitimate or fake. Therefore, test times are also analysed. Although the test times do not provide conclusive evidence about the response time in real time, they can provide a general idea. In the case of real-time adaptation of the proposed model, the response time of the model with 256 neurons will be higher. Models with 32 and 64 neurons can respond quickly, but these networks' classification success is lower than the 128-neuron model. Regarding the proposed model, the optimal number of neurons for classification success and response time is 128.

5.2.6. Results of the classification metrics

After conducting experimental observations, the parameters deemed most suitable for the proposed model were identified. The Adam algorithm was selected as the optimisation algorithm, the BiLSTM was employed to train character and word-based features, and

Table 10
Determination of the number of BiLSTM neurons for the proposed model.

Dataset	BiLSTM Units	Training Accuracy	Validation Accuracy	Test Accuracy	Training Time (sec.)	Test Time (sec.)	Epoch
GramBeddings Dataset [8]	32	0.9874	0.9801	0.9805	20,395.64	131.06	20
	64	0.9914	0.9811	0.9819	23,805.56	141.42	20
	128	0.9936	0.9820	0.9819	29,663.86	169.61	20
	256	0.9929	0.9816	0.9814	50,799.30	293.31	20
	32	0.9978	0.9929	0.9933	6985.26	62.92	20
Malicious and Benign URLs Dataset [22]	64	0.9973	0.9931	0.9933	7591.15	82.63	20
	128	0.9978	0.9932	0.9933	8190.86	82.45	20
	256	0.9977	0.9915	0.9915	12,073.76	84.68	20
	32	0.9933	0.9806	0.9802	1048.38	5.95	20
	64	0.9948	0.9818	0.9822	1085.79	6.13	20
Ebbu2017 Phishing Dataset [2]	128	0.9968	0.9834	0.9831	1229.67	7.15	20
	256	0.9972	0.9803	0.9807	1698.19	9.87	20

the skip-gram algorithms were utilised for the Word2Vec model. The number of neurons in the BiLSTM algorithm was 128, and the maximum URL length was 300 for character-based features. The proposed model is supported by the attention, max pooling, and dropout layers of the BiLSTM algorithm, as illustrated in Fig. 10 and 11. The proposed model was trained, validated, and tested using the GramBeddings, Malicious and Benign URLs, and Ebbu2017 Phishing datasets. The training process was conducted over 30 epochs. The initial dataset partitioning involved an 80% training and 20% testing split. Subsequently, the training data was split into 80% training and 20% validation subsets. The rationale behind this approach is to prevent overlearning and enhance the model's generalisation capability, ensuring that it can correctly classify a URL address it encounters for the first time. For each data set, separate training-validation accuracy and training-validation loss graphs were obtained. The results of the relevant graphs for GramBeddings, Malicious and Benign URLs, and the Ebbu2017 Phishing datasets are presented in Fig. 13, 14, and 15, respectively. The graphs in Fig. 13, 14, and 15 demonstrate that the proposed model exhibits no issues such as overfitting or underfitting in all three datasets.

Fig. 13 illustrates that the optimal validation accuracy value is 0.9829 at epoch 12, while the minimal validation loss value is 0.0515 at epoch 8 when the proposed model is trained with the GramBeddings dataset.

Fig. 14 illustrates that the optimal validation accuracy value is 0.9935, attained at the 30th epoch, while the minimum validation loss value is 0.0258, recorded at the 7th epoch, as a consequence of training the proposed model on the Malicious and Benign URLs dataset.

Fig. 15 demonstrates that the optimal validation accuracy is 0.9855 at epoch 27, while the minimal validation loss is 0.0623 at epoch 7, resulting from training the proposed model with the Ebbu2017 Phishing dataset.

The confusion matrices for the proposed model are presented in Fig. 16. Fig. 16 (a) presents the results of the GramBeddings dataset, and Fig. 16 (b) displays those of the Malicious and Benign URLs dataset. At the same time, Fig. 16 (c) depicts the Ebbu2017 Phishing dataset. The TP, TN, FP, and FN values are presented numerically and in percentages in the confusion matrices. Upon examination of the confusion matrices presented in Fig. 16, it becomes evident that the GramBeddings and Ebbu2017 Phishing datasets exhibit balanced rates. In contrast, the Malicious and Benign URLs dataset displays an imbalanced distribution of values. This is due to the distribution of the label values of the URL addresses in the data sets.

Table 11 illustrates the results obtained from the confusion matrices for each data set, including precision, recall, F1-score and AUC. As indicated in Table 11, the highest value was obtained with a test accuracy value of 0.9824 in the model with a maximum URL length of 300 in the GramBeddings dataset. The AUC value obtained with the GramBeddings dataset was 0.9977. In the Malicious and Benign URLs dataset, the highest value was obtained with a test accuracy value of 0.9937 in the model with a maximum URL length of 250. In the Malicious and Benign URLs dataset, the AUC value was found to be 0.9986 in all models where the URL length was 250, 300 and 350. In the Ebbu2017 Phishing dataset, the highest values were obtained with a test accuracy of 0.9846 and an AUC value of 0.9984 in the model, where the maximum URL length was 350.

Table 11 presents the class column, which displays the labels used in the dataset. 0 represents a legitimate item, while 1 represents a phishing item. The details of the precision, recall, and F1-score classification metrics calculated for both classes are presented in Table 11. As illustrated in Table 11, the ROC curves and AUC values obtained following the training and testing of the model with a maximum URL length of 300 in all data sets are presented in Fig. 16 (d) on the same graph. As demonstrated in Fig. 16 (d), the ROC graphs of the three data sets exhibit a close alignment to the left side and upward trajectory. The ROC curves indicate that the proposed model achieves high classification performance when evaluated across various datasets. The AUC values are between 0.9 and 1.0. The AUC values suggest that the proposed model operates with a significant level of accuracy.

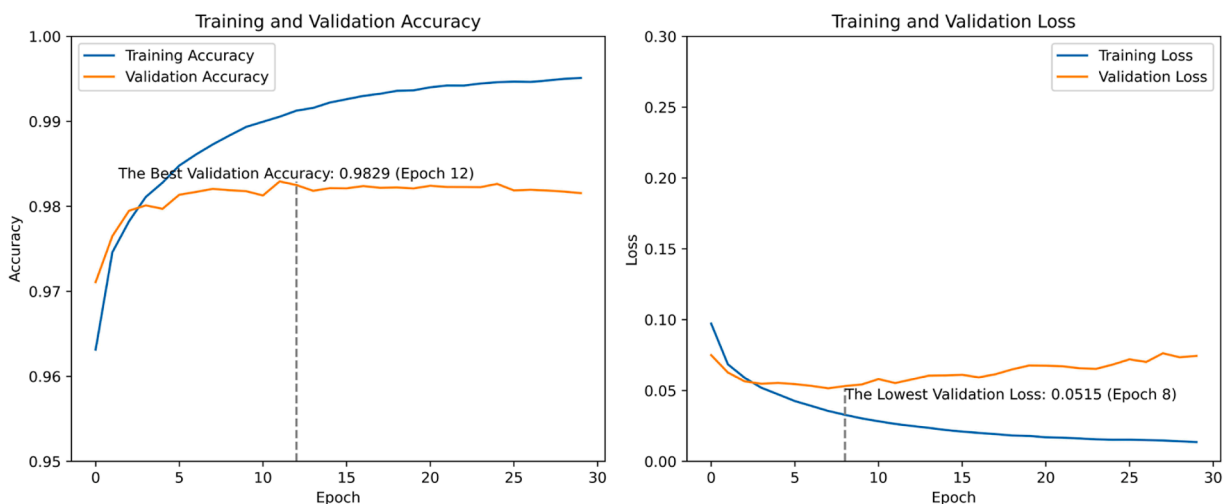


Fig. 13. Graphs depicting the proposed model's training validation accuracy and loss on the GramBeddings dataset.

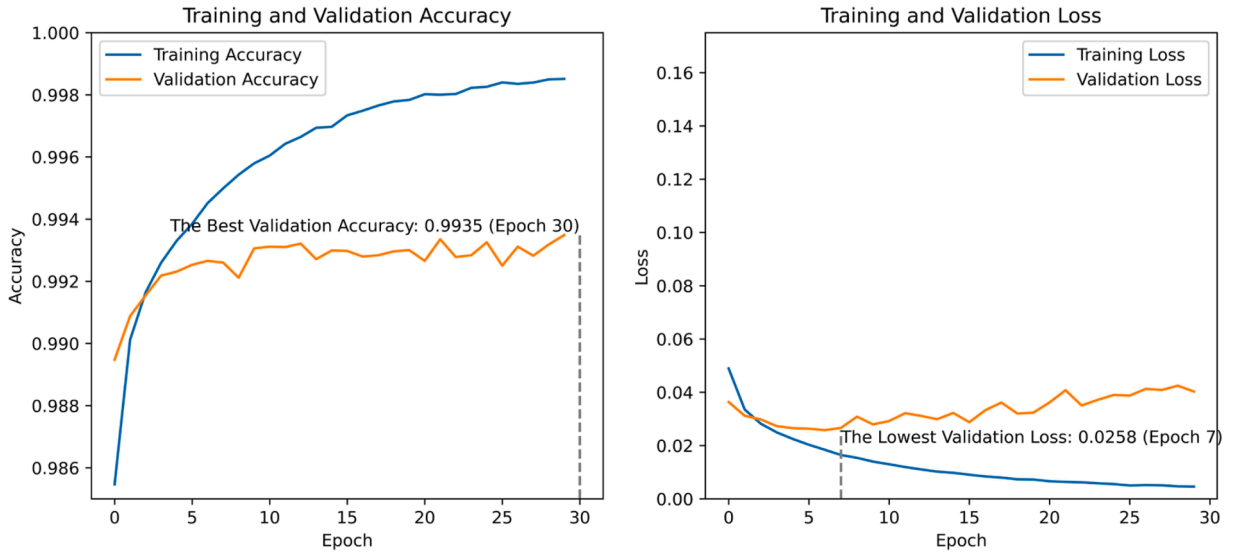


Fig. 14. Graphs depicting the proposed model's training validation accuracy and loss on the Malicious and Benign URLs dataset.

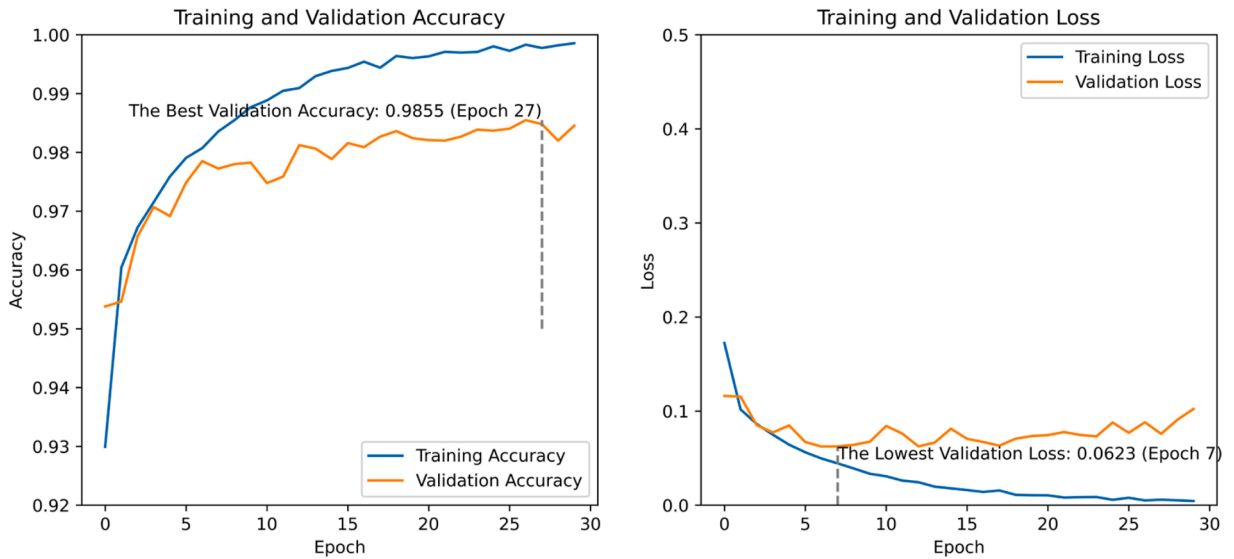


Fig. 15. Graphs depicting the proposed model's training validation accuracy and loss on the Ebbu2017 Phishing dataset.

5.3. URL shortening

A shortened URL is a URL that has been shortened and made easier to use. This is achieved through the use of online services that are employed to create shortened versions of long and complex URLs. To illustrate, a URL shortening service can transform a URL such as <https://www.abc.com/1234?xyz=1a3xpNOQ1xm3> into a shorter address, for example, <https://tinyurl.com/3b4tf5>. Shortened URLs can redirect users to websites that may be harmful or malicious. A statistical analysis was conducted on shortened URLs in three distinct data sets to evaluate the efficacy of the proposed model on shortened URLs. Statistical data on shortened URLs in these datasets are presented in Table 12. The proportion of shortened URL addresses in the GramBeddings dataset is 5.23%, in the Malicious and Benign URLs dataset, it is 5.90%, and in the Ebbu2017 Phishing dataset, it is 6.01%. All shortened URL addresses from the three datasets were consolidated into a unified dataset, designated as the 'Shortened URL Dataset'. This consolidated dataset encompasses 72,854 shortened URLs in total. The dataset comprises 22,626 phishing URLs and 50,228 legitimate URLs, indicating an imbalance in distribution. This imbalance reflects the situation in real-life internet usage. The proposed model was trained and tested using this dataset, and the results demonstrate that the model can classify shortened URL addresses with 92.16% accuracy. Table 13 illustrates the classification metric results obtained from the training and testing process with the URL Shortening Dataset. The maximum number

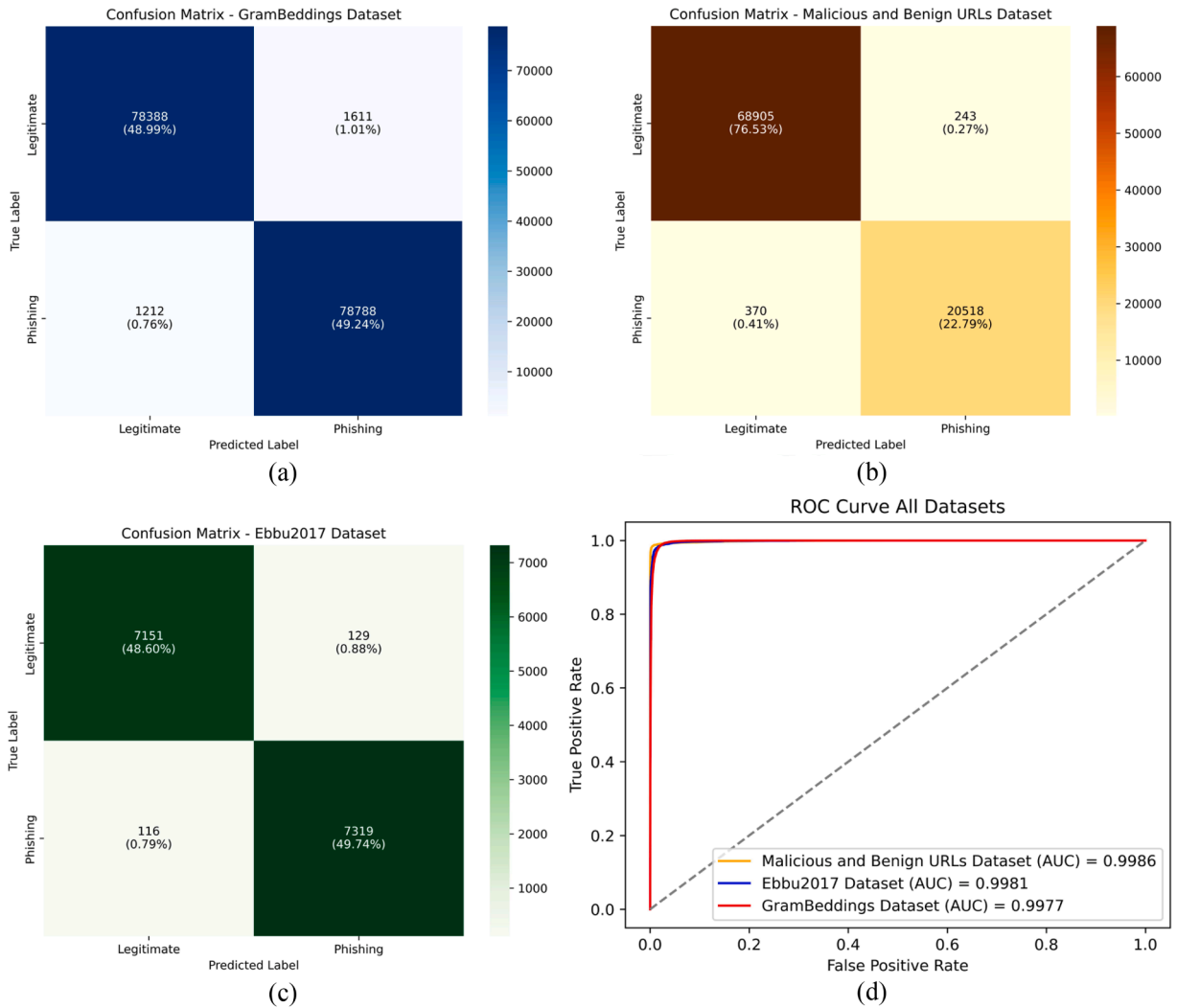


Fig. 16. (a) GramBeddings Dataset confusion matrix, (b) Malicious and Benign URLs Dataset confusion matrix, (c) Ebbu2017 Phishing Dataset confusion matrix, (d) ROC curve and AUC values for all datasets.

Table 11

Classification metric values of the proposed model at different URLs lengths.

Dataset	URL Length	Test Accuracy	Class	Precision	Recall	F1-Score	AUC
GramBeddings Dataset [8]	250	0.9820	0	0.9838	0.9802	0.9820	0.9977
			1	0.9802	0.9838	0.9820	
	300	0.9824	0	0.9848	0.9799	0.9823	0.9977
			1	0.9800	0.9849	0.9824	
	350	0.9809	0	0.9825	0.9792	0.9809	0.9975
			1	0.9793	0.9826	0.9809	
Malicious and Benign URLs Dataset [22]	250	0.9937	0	0.9949	0.9969	0.9959	0.9986
			1	0.9898	0.9831	0.9864	
	300	0.9932	0	0.9947	0.9965	0.9956	0.9986
			1	0.9883	0.9823	0.9853	
	350	0.9927	0	0.9949	0.9957	0.9953	0.9986
			1	0.9856	0.9831	0.9843	
Ebbu2017 Phishing Dataset [2]	250	0.9836	0	0.9832	0.9835	0.9834	0.9983
			1	0.9839	0.9836	0.9837	
	300	0.9834	0	0.9840	0.9823	0.9832	0.9981
			1	0.9827	0.9844	0.9835	
	350	0.9846	0	0.9834	0.9854	0.9844	0.9984
			1	0.9857	0.9837	0.9847	

Table 12

Statistical data on the number of shortened URLs in the three datasets.

	GramBeddings Dataset [8]	Malicious and Benign URLs Dataset [22]	Ebbu2017 Phishing Dataset [2]
Total number of shortened URLs	41,849	26,582	4423
Number of shortened phishing URLs	15,790	5044	1792
Number of shortened legitimate URLs	26,059	21,538	2631
Shortened URL rate	5.23%	5.90%	6.01%
Phishing rate	37.73%	18.98%	40.52%
Legitimate rate	62.27%	81.02%	59.48%

Table 13

Classification metric values of the proposed model at Shortened URL Dataset.

Dataset	URL Length	Test Accuracy	Class	Precision	Recall	F1-Score	AUC	Epoch
Shortened URL Dataset	300	0.9216	0	0.9589	0.9261	0.9422	0.9750	30
			1	0.8476	0.9118	0.8785		

of tokens in the data set is 471. The model's character- and word-based approach enabled the identification of distinctive patterns in the features, thereby providing an effective solution for classifying shortened URL addresses. As no training and testing with shortened URL addresses has been conducted in other studies in the literature, a direct comparison of the results of the current study with those of other studies is not possible. Furthermore, this study's data set obtained from shortened URL addresses is unique.

Table 13 demonstrates that the proposed model, which incorporates the URL Shortening Dataset, exhibits an accuracy rate of 92.16% in classifying shortened URLs. The precision, recall, and F1 scores for legitimate URLs were 95.89%, 92.61%, and 94.22%, respectively. In contrast, for phishing URLs, the precision, recall, and F1-scores were 84.76%, 91.18%, and 87.85%, respectively. The model exhibits a high degree of discriminatory power, as evidenced by an AUC value of 0.9750. The results demonstrate the model's effective and reliable capacity for classifying shortened URLs.

5.4. Unified tokenisation

The GramBeddings, Malicious and Benign URLs and Ebbu2017 Phishing datasets were merged to create a common dictionary. During this process, duplicate tokens were removed and organised to include each token only once. The resulting common word dictionary contains a total of 1320,097 words. The combined dictionary was then used to evaluate the three datasets with the proposed model in the training and testing phases. This study applied separate tokenisation processes to data from three datasets, resulting in various vocabulary sets. This situation may require more clarification on the fair evaluation of models and real-world scenarios. To address this issue, a unified word dictionary encompassing the tokens from the datasets was created, and the proposed model was retrained and tested with this vocabulary across the three datasets. This approach provides strong evidence for the model's effectiveness and ability to adapt to different data sources by ensuring more consistent and generalisable performance across diverse data sources. Table 14 illustrates the classification metric values obtained using the combined tokenisation method for three datasets. The GramBeddings Dataset's test accuracy is 97.95%, while the AUC value is 0.9974. The test accuracy for the Malicious and Benign URLs Dataset is 99.16%, and the AUC value is 0.9983. The test accuracy for the Ebbu2017 Phishing Dataset is 97.86%, with an AUC value of 0.9975. The classification metrics for all other values are provided in detail in Table 14 for both classes. These results demonstrate that the proposed method is a generalisable and practical approach.

5.5. Cost analysis

Table 15 summarises the total, trainable, and non-trainable parameter counts, along with the memory usage values for the datasets. The trainable parameter count and memory usage are identical across all datasets. The developed deep learning model employs the same architecture for each dataset. The model architecture determines trainable parameters and is not affected by the size or type of the dataset.

Pre-trained embedding layers like Word2Vec significantly increase the memory usage of the model, while Bidirectional LSTM and attention layers raise computational requirements and costs. Additionally, metric evaluations during the training process extend processing time and energy consumption. However, combining character and word-level features, the structure provides a significant advantage in detecting phishing attacks and enhancing accuracy and security. Considering the substantial benefits provided, this higher cost is deemed acceptable in critical fields such as finance and security.

5.6. Cross-validation

In the proposed model, the k-fold cross-validation method was evaluated with the Mendeley Data Web Page Phishing Detection

Table 14

Classification metric values were obtained using the unified tokenisation method.

Dataset	Test Accuracy	Class	Precision	Recall	F1-Score	AUC	Epoch
GramBeddings Dataset [8]	0.9795	0	0.9856	0.9733	0.9794	0.9974	20
Malicious and Benign URLs Dataset [22]	0.9916	1	0.9737	0.9858	0.9797	0.9983	20
		0	0.9942	0.9949	0.9946		
Ebbu2017 Phishing Dataset [2]	0.9786	1	0.9832	0.9809	0.9820	0.9975	20
		0	0.9813	0.9754	0.9784		
		1	0.9761	0.9818	0.9789		

Table 15

Cost analysis of model parameters and memory usage across datasets.

Datasets	Total Parameters		Trainable params		Non-trainable params	
	Number of parameters	Model storage size (MB)	Number of parameters	Model storage size (MB)	Number of parameters	Model storage size (MB)
GramBeddings Dataset [8]	110.525.313	421.62	578.433	2.21	109.946.880	419.41
Malicious and Benign URL Dataset [22]	6.3491.713	242.20	578.433	2.21	62.913.280	240.00
Ebbu2017 Phishing Dataset [2]	9.316.225	35.54	578.433	2.21	8.737.792	33.33
Mendeley Data Web Page Phishing Detection Dataset[23]	3.358.593	12.81	578.433	2.21	2.780.160	10.61

dataset, a small dataset due to the occurrence of RAM overload problems in large datasets. The validation was performed with $k = 3, 5$ and 7 , with the highest performance achieved with $k=7$.

Table 16

The byte pair encoding (BPE) method is a compression technique that combines the most frequently occurring character pairs to form subunits of the text, focusing less on semantic relationships than the word-based method. The word-based tokenisation method was preferred in this study to obtain features that focus on semantic relationships in URL addresses.

6. Discussion

Tables 17 and 18 compare the model proposed in this study and other phishing classification studies in the literature, utilising the data sets employed in this study. As seen in the related works section in Section 2, Table 17, and Table 18, the studies on the classification of URL addresses of web pages have yielded successful results. Upon closer examination of Table 17, it becomes evident that Bozkir et al. [8] introduced the GramBeddings dataset to the literature and employed it in their studies, proposing a model with an accuracy of 98.27% and an AUC value of 0.9973. The results obtained by our model in the same dataset were 98.24% accuracy and an AUC value of 0.9977. Upon analysis of the Bozkir et al. [8] studies show that the proposed model exhibited a training accuracy of 0.9935. The accuracy value obtained by our proposed model with the training data set is 0.9951 (Table 9, max_url_length=300). Although the test data set of the model proposed by Bozkir et al. [8] is higher, the area under our proposed model's curve (AUC) value is higher. This demonstrates that our proposed model offers a comparable, effective, and consistent solution for classifying phishing URLs in the GramBeddings dataset, which aligns with previous studies' findings.

Both Roy et al. [19] and Afzal et al. [11] utilised the Malicious and Benign URLs Dataset during their respective studies. Compared to the outcomes of the model proposed by Roy et al. [19], which utilised the BiLSTM algorithm, the outcomes of our proposed model demonstrated better performance. While Roy et al. [19] achieved 99.00% accuracy, the classification success of our proposed model in the same dataset is 99.32%. The researchers did not include the AUC value in their studies, so it was impossible to compare in terms of AUC.

In a separate study utilising the Malicious and Benign URLs Dataset, Afzal et al. [11] achieved an accuracy value of 98.30% and an AUC value of 0.9935 in the proposed model employing an LSTM algorithm. In the same dataset, the accuracy value of our proposed model is 99.32%, while the AUC value is 0.9986. The results indicate that the success rate of our proposed model in classification is

Table 16

A comparison of the proposed model with cross-validation and tokenisation methods.

	k-fold cross validation			Character and word tokenisation	Character and byte pair encoding
	k=3	k=5	k=7		
Test Accuracy	0.9072	0.9076	0.9120	0.9033	0.9212
AUC	0.9708	0.9711	0.9682	0.9694	0.9720
Training Time (sec.)	316.65	318.95	461.23	306.52	597.29
Testing Time (sec.)	2.63	2.66	2.62	1.93	1.91
Epoch	30	30	30	30	30

Table 17

Comparison of the proposed model with other studies in the literature.

Dataset	Related Works	Algorithm	Test Accuracy (%)	AUC
GramBeddings Dataset [8]	Bozkir et al. [8]	CNN- BiLSTM	98.27%	0.9973
	Proposed Model	BiLSTM	98.24%	0.9977
Malicious and Benign URLs Dataset [22]	Roy et al. [19]	BiLSTM	99.00%	N/A
	Afzal et al. [11]	LSTM	98.30%	0.9935
		K-Means Clustering	99.70%	N/A
	Proposed Model	BiLSTM	99.32%	0.9986
Ebbu2017 Phishing Dataset [2]	Sahingoz et al. [2]	Random Forest	97.98%	N/A
	Proposed Model	BiLSTM	98.34%	0.9981
Common Crawl, Alexa, Yandex, PhishTank Mendeley Data	Aslam et al. [20]	LSTM and XGBoost	96.04%	0.9581
	Zaimi et al. [21]	XGBoost	97.82	N/A

Table 18

Comparison of the proposed model with other studies in the literature.

Dataset	Related Works	Algorithm	Class	Precision	Recall	F1-Score
GramBeddings Dataset [8]	Bozkir et al. [8]	CNN- BiLSTM	-	0.9894	0.9759	0.9826
	Proposed Model	BiLSTM	0	0.9848	0.9799	0.9823
			1	0.9800	0.9849	0.9824
Malicious and Benign URLs Dataset [22]	Roy et al. [19]	BiLSTM	0	0.9900	0.9900	0.9900
			1	0.9900	0.9900	0.9900
	Afzal et al. [11]	LSTM	-	0.9900	0.9770	0.9830
		K-Means Clustering	-	0.9700	0.9570	0.9680
Ebbu2017 Phishing Dataset [2]	Proposed Model	BiLSTM	0	0.9947	0.9965	0.9956
			1	0.9883	0.9823	0.9853
	Sahingoz et al. [2]	Random Forest	-	0.9700	0.9900	0.9800
			0	0.9840	0.9823	0.9832
			1	0.9827	0.9844	0.9835

higher. Similarly, Afzal et al. [11] achieved an accuracy value of 99.70% with the K-Means clustering algorithm on the same data set, though they did not include the AUC value obtained with K-Means clustering in their study. The precision value of the model they proposed with K-Means clustering is 0.9700, the recall value is 0.9570, and the F1-score value is 0.9680. The precision value obtained by our proposed model at a maximum URL length of 300 is 0.9947 for legitimate URLs and 0.9883 for phishing URLs. The recall value for legitimate URLs was 0.9965, while the recall value for phishing URLs was 0.9823. The F1-score is 0.9956 for legitimate and 0.9853 for phishing. In terms of the precision, recall, and F1-score metrics, it can be observed that our proposed model demonstrates better classification success.

Sahingoz et al. [2] introduced the Ebbu2017 Phishing dataset to the literature, achieving an accuracy rate of 97.98% using the Random Forest algorithm. With the Ebbu2017 Phishing dataset, our proposed model achieved an accuracy rate of 98.34%. It can be observed that the classification success of our proposed model is higher. Sahingoz et al. [2] did not include the AUC value in their study, thus precluding any comparison regarding this metric.

Although the datasets in [20] were obtained from publicly available platforms, they were not fully accessible in the exact form used by the researchers in their studies. Consequently, it was impossible to perform experimental observations using the proposed model. Similarly, the researchers in [21] used a dataset with pre-extracted features. However, since the URLs from which the extracted features were not included as raw data in the dataset used in [21], and our proposed model is designed to extract features directly from raw data, testing our model on this dataset was impossible. Nevertheless, when evaluating the results of our proposed model on other commonly used datasets in the literature, it is observed that our model outperforms the studies in [20] and [21].

Previous research utilising identical data sets has demonstrated the effectiveness of the proposed model for addressing the phishing URL classification issue. The proposed model offers a highly effective solution for detecting phishing URLs.

The model proposed in this study offers an innovative approach to detecting phishing attacks by combining character and word-based feature extraction methods and using BiLSTM and attention mechanisms to improve accuracy. Due to reliance on fixed rules, heuristic methods are insufficient to adapt to new and sophisticated phishing techniques. In contrast, the proposed model demonstrates high generalisation performance when trained on large datasets, thanks to its learning-based structure. Traditional machine learning methods (e.g. Naive Bayes and Random Forest) often rely on manually selected features, which limits their performance. The proposed model overcomes this limitation by automatically extracting and integrating character and word-based features. As shown in Table 17, our model outperforms traditional methods, such as K-Means clustering [11] and Random Forest [2]. These results indicate that the proposed model performs better than traditional methods.

Table 18 illustrates the efficacy of the proposed model on disparate datasets compared to existing literature. Compared with three datasets (GramBeddings, Malicious and Benign URLs, Ebbu2017 Phishing) and related works, the proposed model generally exhibits superior test precision, recall, and test F1-score values. A comparison of the results of the proposed model with other studies utilising similar algorithms and disparate methodologies (CNN-BiLSTM, LSTM, Random Forest) reveals that the proposed model consistently

exhibits superior or comparable performance. This provides compelling evidence of the efficacy and competitive performance of the proposed model. The experimental findings presented in [Section 5](#) and the comparison of the proposed model with existing studies in the literature allow the following conclusions to be drawn about the proposed model:

1. The deep learning model obtained by combining character and word-based feature extraction methods provides an effective solution for classifying phishing URLs.
2. Since both character and word-level feature extraction methods are used and the extracted features are combined, rich features are obtained.
3. Given that character-based feature extraction is concerned with the analysis of characters, the consideration of semantic relations is not a focus. In addition, word-based feature extraction enabled the study of semantic relations.
4. The proposed method exhibits high classification performance across datasets with varying URL addresses, including balanced (GramBeddings and Ebbu2017 Phishing Datasets) and unbalanced (Malicious and Benign URLs Dataset) datasets.
5. A distinctive dataset comprising abbreviated URL addresses was employed. The model's character and word-based methodology provided an efficacious solution for classifying these URLs.
6. A unified lexicon covering tokens from three different datasets was created to provide a model that can be applied in real-world scenarios. The results demonstrated the effectiveness of the proposed model.
7. Experiments conducted on the Mendeley Data Web Page Phishing Detection dataset have shown that in the absence of memory problems, training and validation can be successfully performed using the cross-validation method.

6.1. Strengths, limitations and future work of the model

The proposed model demonstrates high performance in URL classification by combining character and word-based feature extraction methods. Character-based features focus on superficial properties, while word-based features analyse word relationships to provide a deeper contextual understanding. Integrating these two distinct feature sets enables a comprehensive analysis that captures URLs' structural and semantic characteristics. The model analyses each URL separately at the character and word level, generating two distinct sets of features. The character-based analysis evaluates individual characters within the URL, while the word-based analysis examines the contextual relationships between words. These two feature sets are then integrated into the model's concatenate layer to create a single feature set for classification. This architecture supports independent feature analysis and preserves the strengths of different feature types, resulting in more effective classification. The robust design of the model has been tested on various datasets, including GramBeddings, Malicious and Benign URLs, Ebbu2017 Phishing, and Mendeley Data Web Page Phishing Detection, demonstrating resilience to scalability and performance challenges.

One of the model's strengths is its ability to analyse URLs at both character and word levels. Processing these two levels independently compensates for the limitations of each approach. For example, while character-based features do not focus on semantic relationships, word-based features emphasise them. In addition, word-based feature extraction generates tokens based on characters such as “.”, “/” without evaluating whether these characters indicate phishing attributes. However, character-based features explicitly consider these characters, ensuring that features missed by one method are captured by the other. This two-layered analysis dramatically enhances the model's ability to identify complex URL patterns. Despite its strengths, the model has some limitations and weaknesses. One of these is the variability of word lengths, which can affect classification performance. Tokenising URLs based on characters such as “.”, “/” results in word lengths ranging from one character to multiple characters. While spaces typically separate tokens in written texts such as books or articles, the concatenated structure of URLs inherently introduces variability in word lengths. This variability, particularly the lack of a defined range of word lengths, is considered a weakness. Furthermore, representations generated by frequency-based methods such as Word2Vec, Bag of Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) can lead to overloading and memory problems due to the large number of tokens in the dictionary. This highlights the need for optimisation strategies to improve memory efficiency. In addition, the model's effectiveness in real-time environments was not evaluated, as this was outside the scope of the study.

Future research should focus on addressing these limitations through targeted improvements. Techniques such as quantisation and pruning can be used to address memory and performance issues. Caching strategies for pre-processed URLs and assigning appropriate models for URLs with different priority levels can also help to balance cost and performance. Developing an algorithm to control word length could further improve the robustness of the model. Still, such an algorithm should differ from traditional character or word-based n-gram methods. Whereas character n-grams focus on a fixed number of preceding or succeeding characters, word n-grams concentrate on a fixed number of preceding or succeeding words. Given the inherent variability in URL tokenisation processes, traditional n-gram methods may not be appropriate. Therefore, innovative approaches tailored to the unique structure of URLs are required. Real-time detection is another critical area for future research. Evaluating the performance and response time of the model in real-time environments is essential to understanding how it handles previously unseen URLs. For example, complex models could be reserved for critical URLs, while simpler models could be applied to low-priority URLs. These strategies can improve the model's performance, making it more suitable for large datasets and real-time applications. Such enhancements will strengthen the model's adaptability and effectiveness in detecting phishing attacks in a constantly evolving threat landscape.

7. Conclusion

This study proposes a deep learning model for detecting phishing attacks. The model analyses URL data using character and word-based feature extraction methods to classify phishing and legitimate addresses accurately. Experimental results on GramBeddings, Malicious and Benign URLs, and Ebbu2017 Phishing show that the model achieves high accuracy, precision, and AUC values on balanced and unbalanced datasets. Furthermore, experiments on the Mendeley Data Web Page Phishing Detection dataset show the model's effectiveness on small datasets and suggest that cross-validation could be implemented if memory constraints are addressed. The model uses character-based feature extraction to capture the structural details of URLs and word-based feature extraction to assess their semantic content. This dual approach allows the model to analyse phishing attacks at character and word levels, allowing one method to detect attacks that the other may miss. As a result, the model increases fault tolerance and improves its ability to detect complex and sophisticated phishing strategies. Supported by a BiLSTM algorithm and an attention mechanism, the model transforms high-dimensional features into meaningful and dense vectors, optimising the learning process and eliminating the need for manual feature engineering. The model's language independence allows it to work effectively on URLs with different language and character structures without relying on third-party services. It also reduces storage costs and training time by converting high-dimensional characters and word vectors into low-dimensional representations. Experimental results confirm that the proposed model outperforms existing methods in phishing URL detection, achieving high precision and recall rates. The model is a reliable defence mechanism against phishing attacks, contributing to enhanced Internet security and user protection. It provides a sustainable and effective solution to the evolving nature of phishing threats.

CRedit authorship contribution statement

Öznur Şifa Akçam: Conceptualization, Data curation, Methodology, Resources, Software, Visualization, Writing – original draft. **Adem Tekerek:** Conceptualization, Formal analysis, Investigation, Project administration, Supervision, Resources, Validation, Writing – review & editing. **Mehmet Tekerek:** Project administration, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] Ozcan A, Catal C, Donmez E, Senturk B. A hybrid DNN-LSTM model for detecting phishing URLs. *Neural Comput. Appl.* 2023;35(7):4957–73. <https://doi.org/10.1007/s00521-021-06401-z>.
- [2] Sahingoz OK, Buber E, Demir O, Diri B. Machine learning based phishing detection from URLs. *Expert. Syst. Appl.* 2019;117:345–57. <https://doi.org/10.1016/j.eswa.2018.09.029>.
- [3] APWG. 2023. Phishing activity trends report. https://docs.apwg.org/reports/apwg_trends_report_q4_2023.pdf (accessed 7 June 2024).
- [4] Do NQ, Selamat A, Krejcar O, Herrera-Viedma E, Fujita H. Deep Learning for phishing detection: taxonomy, current challenges and future directions. *IEEE Access.* 2022;10:36429–63. <https://doi.org/10.1109/ACCESS.2022.3151903>.
- [5] Aljofey A, Jiang QS, Qu Q, Huang MQ, Niyigena JP. An effective phishing detection model based on character level convolutional neural network from URL. *Electronics.* (Basel) 2020;9(9):1514. <https://doi.org/10.3390/electronics9091514>.
- [6] Zhang XC, Lashkari AH, Ghorbani AA. Classifying and clustering malicious advertisement uniform resource locators using deep learning. *Comput. Intell.* 2021;37(1):511–37. <https://doi.org/10.1111/coin.12422>.
- [7] Tang L, Mahmoud QH. A survey of machine learning-based solutions for phishing website detection. *Mach. Learn. Knowl. Extrac.* 2021;3(3):672–94. <https://doi.org/10.3390/make3030034>.
- [8] Bozkir AS, Dalgic FC, Aydos M. GramBeddings: A new neural network for URL based identification of phishing web pages through N-gram embeddings. *Comput. Secur.* 2023;124:102964. <https://doi.org/10.1016/j.cose.2022.102964>.
- [9] Zheng F, Yan Q, Leung VCM, Yu FR, Ming Z. HDP-CNN: highway deep pyramid convolution neural network combining word-level and character-level representations for phishing website detection. *Comput. Secur.* 2022;114:102584. <https://doi.org/10.1016/j.cose.2021.102584>.
- [10] Wang, W.P., Zhang, F., Luo, X., Zhang, S.G., 2019. PDRCNN: precise phishing detection with recurrent convolutional Neural Net. *Secur. Commun. Net.*, 2019, <https://doi.org/10.1155/2019/2595794>.
- [11] Afzal S, Asim M, Javed AR, Beg MO, Baker T. URLdeepDetect: A deep learning approach for detecting malicious URLs using semantic vector models. *J. Net. Syst. Manag.* 2021;29(3). <https://doi.org/10.1007/s10922-021-09587-8>.
- [12] Benavides-Astudillo E, Fuertes W, Sanchez-Gordon S, Nuñez-Agurto D, Rodríguez-Galán G. A phishing-attack-detection model using natural language processing and Deep learning. *Appl. Sci.* 2023;13(9):5275. <https://doi.org/10.3390/app13095275>.
- [13] Elsadig M, Ibrahim AO, Basheer S, Alohal MA, Alshunaifi S, Alqahtani H, Alharbi N, Nagmeldin W. Intelligent deep machine learning cyber phishing URL detection based on BERT features extraction. *Electronics.* (Basel) 2022;11(22):3647. <https://doi.org/10.3390/electronics11223647>.
- [14] Yang P, Zhao GZ, Zeng P. Phishing website detection based on multidimensional features driven by deep learning. *IEEE Access.* 2019;7:15196–209. <https://doi.org/10.1109/ACCESS.2019.2892066>.
- [15] Alshehri M, Abugabah A, Algarni A, Almotairi S. Character-level word encoding deep learning model for combating cyber threats in phishing URL detection. *Comput. Electr. Eng.* 2022;100:107868. <https://doi.org/10.1016/j.compeleceng.2022.107868>.
- [16] Rasymas T, Dovydaits L. Detection of phishing URLs by using deep learning approach and multiple features combinations. *Baltic J. Modern Comput.* 2020;8(3):471–83. <https://doi.org/10.22364/bjmc.2020.8.3.06>.

- [17] Rao RS, Vaishnavi T, Pais AR. PhishDump: A multi-model ensemble based technique for the detection of phishing sites in mobile devices. *Pervasive Mob. Comput.* 2019;60:101084. <https://doi.org/10.1016/j.pmcj.2019.101084>.
- [18] Xiao X, Xiao WT, Zhang DY, Zhang B, Hu GW, Li Q, Xia ST. Phishing websites detection via CNN and multi-head self-attention on imbalanced datasets. *Comput. Secur.* 2021;108:102372. <https://doi.org/10.1016/j.cose.2021.102372>.
- [19] Roy SS, Awad AI, Amare LA, Erkihun MT, Anas M. Multimodel phishing URL detection using LSTM, bidirectional LSTM, and GRU models. *Future Internet*, 2022; 14(11):340. <https://doi.org/10.3390/fi14110340>.
- [20] Aslam S, Aslam H, Manzoor A, Chen H, Rasool A. AntiPhishStack: LSTM-based stacked generalization model for optimized phishing URL detection. *Symmetry*. (Basel) 2024;16(2):248. <https://doi.org/10.3390/sym16020248>.
- [21] Zaimi R, Hafidi M, Lamia M. A deep learning mechanism to detect phishing URLs using the permutation importance method and SMOTE-tomek link. *J. Supercomput.* 2024;80(12):17159–91. <https://doi.org/10.1007/s11227-024-06124-7>.
- [22] Kumar, S., 2019. Malicious and benign URLs. <https://www.kaggle.com/datasets/siddharthkumar25/malicious-and-benign-urls>. (accessed 13 May 2024).
- [23] Hannousse, A., Yahiouche, S., 2021. Web page phishing detection, Mendeley Data. V3. [10.17632/c2gw7fy2j4.3s](https://doi.org/10.17632/c2gw7fy2j4.3s).
- [24] Asiri S, Xiao Y, Alzahrani S, Li SH, Li TS. A survey of intelligent detection designs of HTML URL phishing attacks. *IEEe Access*. 2023;11:6421–43. <https://doi.org/10.1109/ACCESS.2023.3237798>.
- [25] Wang HH, Yu L, Tian SW, Peng YF, Pei XJ. Bidirectional LSTM malicious webpages detection algorithm based on convolutional neural network and independent recurrent neural network. *Appl. Intel.* 2019;49(8):3016–26. <https://doi.org/10.1007/s10489-019-01433-4>.
- [26] Alshingiti Z, Alaql R, Al-Muhtadi J, Haq QEU, Saleem K, Faheem MH. A deep learning-based phishing detection system using CNN, LSTM, and LSTM-CNN. *Electronics*. (Basel) 2023;12(1):232. <https://doi.org/10.3390/electronics12010232>.
- [27] Ali W, Ahmed AA. Hybrid intelligent phishing website prediction using deep Neural Net. with genetic algorithm-based feature selection and weighting. *IET. Inf. Secur.* 2019;13(6):659–69. <https://doi.org/10.1049/iet-ifs.2019.0006>.
- [28] Zhu EZ, Yuan QX, Chen ZL, Li XJ, Fang XY. CCBLA: A lightweight phishing detection model based on CNN, BiLSTM, and attention mechanism. *Cognit. Comput.* 2023;15(4):1320–33. <https://doi.org/10.1007/s12559-022-10024-4>.
- [29] Xiao X, Zhang DY, Hu GW, Jiang Y, Xia ST. CNN-MHSA: A Convolutional Neural network and multi-head self-attention combined approach for detecting phishing websites. *Neural Net.* 2020;125:303–12. <https://doi.org/10.1016/j.neunet.2020.02.013>.